Semantic Fragmentation and Stochastic Assembly: A Protocol for Decentralized Language-Model Inference over Untrusted Volunteer Nodes

Sebastián A. Espinoza-Ulloa, Ph.D. Independent Researcher ORCID: 0000-0003-1497-356X · GitHub: @Sebastardito
Correspondence: sebas_saeu@hotmail.com

> **Note on affiliation and independence.** This work is carried out entirely in a personal capacity as an independent researcher. The author holds separate academic affiliations — Pontificia Universidad Católica del Ecuador (saespinozau@puce.edu.ec) and University of Saskatchewan (sebastian.espinoza@usask.ca) — that are unrelated to the subject matter of this work. **Neither has provided funding, materials, computing resources, personnel or institutional support for this project, and no institutional endorsement is claimed or implied.**
>
> **Relevant background.** The author holds a Ph.D. in Biology (University of Saskatchewan) in avian population genomics, and works on whole-genome sequencing, variant calling and de novo assembly. The genome-assembly framing used throughout this paper derives from that background; Section 2.4 states explicitly where the analogy holds and where it does not.

Version 1.4 — 14 August 2026 Licence of this document: CC BY 4.0. Reference implementation: AGPL-3.0-or-later. Repository: https://github.com/Sebastardito/Swarmbly-AI

> **Purpose of this document.** This is an *enabling* disclosure, published defensively to establish prior art. It specifies the protocol in sufficient detail for a person skilled in the art to implement it, states its parameters and their derivation, declares the hypotheses on which its viability depends, and reports the evidence that argues *against* it as prominently as the evidence that argues for it. Section 13 lists the disclosed elements as numbered claims.

Abstract

Peer-to-peer language-model inference has so far been pursued by splitting the *model*: transformer layers or tensors are distributed across machines, and intermediate activations traverse the public internet on every generated token. This places the design squarely against a bandwidth gap of roughly five orders of magnitude and a latency gap of four to five, between datacenter interconnect and consumer last-mile links.

This paper specifies **Swarmbly**, a protocol that distributes the *problem* instead. A client-side orchestrator, itself a small language model, decomposes a request into a directed acyclic graph of semantic micro-tasks; each micro-task is dispatched once, asynchronously, to a volunteer node running a complete small model (1–8B parameters); returned fragments — *contigs*, in the genome-assembly vocabulary the design borrows — are verified, selected and spliced locally. Network traversal occurs once per fragment per session rather than once per layer per token.

I make five contributions. First, I identify the **context budget** S — the number of tokens of shared context accompanying each dispatched fragment — as a single scalar on which four design desiderata pull in conflicting directions: assembly coherence and fragment verifiability increase in S, privacy-by-decontextualization decreases in S, and the capability required of a worker model plausibly decreases in S. The protocol's viability reduces to whether a value of S exists that satisfies all four thresholds simultaneously. This is a falsifiable proposition, and I state it as such.

Second, I decompose swarm performance into **coverage** (does any worker produce an acceptable fragment) and **conversion** (does the orchestrator turn acceptable fragments into an acceptable whole), and argue from the published record that coverage scales with node count while conversion does not — bounding the marginal value of an additional node by the client's selection ability, and inverting the usual engineering priority.

Third, I specify the wire protocol, the assembly algorithm, the verification scheme and the parameter derivations in implementable detail.

Fourth, I publish a reference harness that measures the **coherence tax** — quality lost to fragmentation and reassembly — as a function of S, together with an explicit go/no-go criterion under which the architecture should be abandoned. **That measurement has now been made.** Against three model families served locally, the coherence tax falls monotonically in S — 24.1 %, 20.4 %, 16.1 %, 13.7 % across the swept range — and the abandonment criterion is met in three task categories. In two of those the tax is **negative**: fragmenting the problem and reassembling it produced a *better* answer than the monolithic baseline, by as much as 9.0 %. The companion V3c run finds **no relationship between inter-replica agreement and judged quality** ($r = -0.030$ over 597 semantic units), which does not support the confidence map described below; Section 11.3 reports both results in full, including why the second is unsupported rather than refuted.

Fifth, I introduce a second routing axis, orthogonal to content sensitivity: a client-side privacy classifier assigns every request to a **tier** — an open volunteer mesh, a permissioned *trusted swarm* whose membership is a cryptographic public-key whitelist under mutual TLS, or purely local execution — and the same protocol and the same client run at all three.

This is what makes the architecture deployable where an anonymous volunteer cannot lawfully be a data processor, and it separates two roles the replica count k had been serving at once: defence against dishonest workers, which a whitelist removes, and the independent replicas that the confidence map requires, which it does not.

The compute already exists. Volunteer computing platforms today aggregate on the order of 700,000 active devices, four million CPU cores and 560,000 GPUs at an average throughput of 93 PetaFLOPS — from a participant base that has been *declining* for two decades, which makes it a floor rather than a ceiling. What is missing is not silicon. It is a protocol under which that silicon can serve language-model inference without its owners surrendering control, and without a datacenter interconnect.

Swarmbly is that protocol, and its central architectural consequence is that **the barrier to serving AI stops being capital and becomes participation**.

Decentralization also yields one capability that centralization cannot replicate. When k replicas of a micro-task are produced by nodes running *different* model families and aligned against each other, the per-unit agreement between them is a measurable signal, and the protocol returns a **map of low-confidence regions** alongside every answer — the direct analogue of per-base quality in a genome assembly. A monolithic provider running one model has nothing to align. This is an epistemic property that emerges *from* distribution rather than being sacrificed to it.

I state precisely what is not claimed — latency parity, unlimited context, cryptographic confidentiality, a demonstrated carbon benefit — in Section 1.4, and I devote Section 12 to limitations and negative results, because a specification that can be shown to be wrong is worth more than a promise that cannot be checked.

Keywords: decentralized inference · peer-to-peer systems · prompt decomposition · small language models · verifiable computation · volunteer computing · genome assembly

---

1. Introduction

1.1 The concentration is in the capital, not the knowledge

The capability to build language models is no longer scarce. Model weights, training recipes, and inference engines are published openly and improve monthly. What remains scarce — and what concentrates power — is the capital required to *operate* them at scale: the accelerators, the buildings, the power contracts and the interconnect.

That concentration has a measurable shape. Data centres consumed 415 TWh in 2024, roughly 1.5 % of world electricity, with projections to 945 TWh by 2030 [83]. Hyperscale facilities in the United States draw from grids measured at 545 gCO_2/kWh against a 370 g national average [84]. These are the figures of an industry whose growth path runs through construction, and construction is available only to those who can finance it.

The consequence is structural rather than conspiratorial: a technology whose *knowledge* is public becomes, in practice, controllable by whoever can afford the *hardware*. Openness of weights does not democratize a capability whose operation costs hundreds of millions of dollars.

And yet the hardware already exists, distributed and idle. The flagship volunteer-computing platform aggregates approximately 700,000 active devices, 4 million CPU cores and 560,000 GPUs at an average 93 PetaFLOPS [47] — and it does so from a participant base that has shrunk from roughly a million to about two hundred thousand over two decades. That number is a *floor*, drawn from a declining niche, not a projection of what a compelling protocol could mobilise. Measured at the level of the individual node, idle consumer GPUs serve LLM inference at \$0.111–0.149 per million tokens on an RTX 4090, at 62–78 % of H100 throughput for roughly half the cost [49].

The world's spare inference capacity is not a hypothesis. The missing piece is a protocol under which it can be used — and the reason no such protocol exists yet is a physical constraint that the next section states exactly.

1.2 The physical constraint and the reframing

Any architecture for distributed language-model inference over consumer hardware is decided, before any algorithm is chosen, by one measurement. An NVIDIA H100 SXM moves 900 GB/s per GPU over NVLink; Quantum-2 InfiniBand offers 400 Gb/s per port and 51.2 Tb/s aggregate per switch. Typical consumer upstream bandwidth is of the order of 60 Mbps. The ratio is roughly 120,000× against NVLink and 6,700× against InfiniBand. Intra-node NVLink latency is sub-microsecond and InfiniBand single-digit microseconds, against 30–170 ms of wide-area round-trip time: **four to five orders of magnitude** [22, 23, 24].

This single fact partitions the design space cleanly. Architectures that require communication *per token* are pushed against the gap on every step of generation, whereas architectures that cross it *once per unit of work* are not, and everything else in this paper follows from choosing the second class.

The measured behaviour of the first class is consistent with the prediction. Petals — the reference implementation of pipeline-parallel inference over the internet — serves Llama-2-70B on three T4s at 2.29 steps/s over a 1 Gbit/s link with sub-5 ms RTT, falling to 1.57 steps/s at 100 Mbit/s and 100 ms: a 31 % loss attributable to the network alone. A real geodistributed swarm of fourteen heterogeneous servers achieves 0.83 steps/s [1, 2]. Analyses of model-parallel schemes at public-internet latency find that pipeline parallelism is the *only* viable model-parallel arrangement — it communicates

least — and that asynchronous micro-batching does not help, because decoding is bound by KV-cache movement rather than by compute [25].

The conclusion I draw is not that pipeline parallelism was implemented poorly. It is that it is the right answer to the wrong question.

Swarmbly asks a different question: rather than *how to run one large model across many machines*, **how to run many complete small models on one large problem**.

The two are not variations of the same idea. Splitting a model creates a chain in which node k cannot begin until node $k-1$ finishes, and in which every token retraces the chain. Splitting a problem creates a set — more precisely a partial order — in which independent sub-tasks proceed concurrently and each crosses the network once. The first is bounded by $\Sigma_i(t_{compute,i} + t_{net,i})$; the second by $\max_i(t_{compute,i} + t_{net,i})$ plus local assembly. The structural claim is that the second is the regime in which volunteer hardware can participate at all.

The design vocabulary is borrowed deliberately from genome shotgun assembly. A request is fragmented into *reads*; each returned answer is a *contig*; adjacent contigs are joined by *overlap* and, where they disagree, by *consensus*; the plan that orders them is a *scaffold*. I am explicit in Section 2.4 about what this analogy does and does not license: it supplies a vocabulary, a set of failure modes, and one genuinely transferable warning. It does not supply a transferred algorithm, and I make no such claim.

1.3 What this makes possible

The first measurement is now in, and the central prediction held: the coherence tax falls monotonically in the context budget, and in three task categories it clears the abandonment threshold that was fixed before any data existed — in two of them by producing a *better* answer than the monolithic baseline (Section 11.3). One run, at one scale, on eight prompts — one of which produced no usable baseline — does not make a protocol proven, and Section 11 still states the measurement under which I would conclude the design fails. What it does mean is that the falsifiable core of Section 4 survived its first contact with evidence, and that four things follow which are not available today.

1. Serving capacity without owning it. A participant contributes a machine that already exists and already draws power when idle. The entry requirement is a complete small model, not a shard of a large one, which places the addressable hardware pool orders of magnitude above what pipeline-parallel schemes can reach. Capacity then scales with *participation* rather than with capital expenditure — a growth curve that no centralized operator can match, because theirs is bounded by what they can build and finance.

2. A confidence map that centralization structurally cannot produce — a mechanism, not yet a demonstrated benefit. Section 8.4b develops this. It was, in an earlier draft of this paper, described as the most immediately valuable user-facing property of the architecture; the first measurement (Section 11.3) does not support that description and it has been withdrawn. What remains is a mechanism whose value is unmeasured. Because a micro-task is answered by k nodes running *different* model families, the answers can be aligned against each other and the agreement scored per semantic unit. Regions where independent models converge are reported as such; regions where they diverge are surfaced as low-confidence, exactly as an assembler reports per-base quality rather than a uniformly confident sequence. **A provider running one model has nothing to align.** The redundancy that decentralization requires turns out to produce a signal that centralization cannot obtain at any price. **Whether that signal carries information about correctness is a separate question, and the first attempt to measure it came back flat** (Section 11.3). The mechanism is real; its usefulness is unproven, and the experiment that would settle it is specified in Section 11.4.

3. Context bounded by the user's machine rather than by a vendor's product decision. Fragmentation relocates the context limit from a fixed window set by a provider to a function of the client's assembly time and memory. With hierarchical assembly the working memory required grows logarithmically with total volume, so the practical ceiling for a modern personal machine sits far above what an individual user would exhaust — and, unlike a vendor's window, it rises when the user upgrades rather than when a price tier changes.

4. A substrate that can be audited rather than trusted. The protocol, the client, the node software and the licence are public. The share of traffic served by foundation-operated anchor nodes is published (Section 10.4). The energy accounting is published against a public standard (Section 10.3). Coherence degradation is returned with every response rather than concealed (Section 8.6). None of these is a courtesy; each is a conformance requirement in the specification, and an implementation that omits them is non-conformant.

Taken together these describe a different distribution of control over a general-purpose technology — not a cheaper way to buy the same thing. I will not dress that as a modest claim, because it is not one: if the context budget holds at scale, the precondition for *serving* a general-purpose technology stops being a data centre and becomes a laptop and a protocol. Whether that redistribution is achievable is an empirical question, and the rest of this paper is written to make it answerable rather than rhetorical.

1.4 Scope of claims

Four claims a reader might expect here are deliberately absent, and Section 12 develops each in full.

I do not claim **latency parity**: single-node speculative decoding already delivers 2–3× with a proof that the output distribution is preserved [13], and no fragmentation scheme competes with that on speed. The comparison that matters

is different — for a user without the hardware to run a capable model at all, the relevant axis is not *faster or slower* but *possible or impossible*.

I do not claim **unlimited context**, only a relocated and much higher limit (Section 1.3, point 3).

I do not claim **cryptographic confidentiality**. Fragmentation is not encryption, Section 9 gives the attacks that settle it, and the protocol routes sensitive work to local execution or attested hardware instead of pretending otherwise.

I do not claim a **demonstrated environmental benefit**. The embodied-carbon argument is strong and the operational one is conditional; Section 10.3 states both, along with the measurement I commit to publishing whatever it shows.

Stating these plainly costs nothing that was ever real, and it is what allows the claims in Section 1.3 to be read as engineering rather than advertising.

1.5 Contributions and structure

Section 2 surveys the state of the art. Section 3 states design principles. Section 4 develops the **context budget**, the paper's central conceptual contribution. Section 5 formalizes the **swarm-of-small-models** thesis and states the coverage/conversion decomposition. Section 6 gives the architecture, Section 7 the wire protocol, Section 8 the algorithms. Section 9 covers privacy, verification, privacy tiers and adversarial nodes. Section 10 sketches economics and governance. Section 11 describes the reference harness and the evaluation protocol, including the criterion under which I would abandon the design. Section 12 lists limitations and negative results. Section 13 declares the disclosed elements for prior-art purposes.

---

2. Background and related work

2.1 Decentralized inference by model partition

Petals [1, 2] distributes contiguous blocks of transformer layers across volunteers; clients hold embeddings locally and route activations through a chain of servers. I treat it as the field's pioneer rather than as a competitor: it demonstrated that peer-to-peer inference over the public internet is possible at all, which is the precondition for this work. Swarmbly does not improve pipeline parallelism; it declines to use it, and that divergence is a difference of strategy rather than of quality. Petals' last release dates from September 2023 [3]. Hivemind and SWARM parallelism [4, 5] address fault-tolerant training over unreliable heterogeneous devices with the same underlying premise: the model is the unit of distribution.

Bittensor [6, 7] adds an incentive layer, with a consensus mechanism whose connectivity-based regularization is described as resistant to collusion of up to 50 % of network weight — a formulation that, read carefully, presupposes a trust anchor.

2.2 Decentralized training over slow links

The subfield that has advanced most is training, and it advanced by attacking communication volume rather than topology. DiLoCo matches fully synchronous optimization while communicating 500× less [8]. OpenDiLoCo trained across two continents at 90–95 % compute utilization [9]. INTELLECT-1 trained a 10B-parameter model on 1T tokens across up to 14 concurrent nodes on three continents with 30 independent contributors and a 400× bandwidth reduction [10]. Subspace/Protocol Models report matching datacenter model-parallel convergence at 80 Mbps against 100 Gbps [11].

The lesson Swarmbly takes is methodological: the bandwidth problem yields to compression and asynchrony. A protocol that reinvents this rather than adopting it is wasting effort.

2.3 Task-level parallelism

Skeleton-of-Thought (SoT) [12] is the direct precedent for decomposing a *prompt*: a skeleton prompt produces a list of points, each expanded independently and in parallel. It reports up to 2.39× speedup, and — this matters more — it reports its own damage: quality improves on knowledge, generic, common-sense, roleplay and counterfactual questions, and degrades on maths, coding, writing and Fermi estimation; on the coherence metric, SoT "is not worse than normal generation around 60 % of the time," which is to say it *is* worse roughly 40 % of the time. The authors state the structural cause without hedging: "SoT currently ignores the dependencies between points."

Their response was not to defend the method but to gate it. SoT-R [12] adds a router that decides per question whether to decompose at all; a trained 120M RoBERTa router suffices, and it is trained with a Tversky loss precisely to penalize false positives — encoding the asymmetry that wrongly fragmenting is worse than wrongly declining to.

Descendants refine the idea. APAR [16] has the model plan its own parallel branches. PASTA [17] learns an annotation language for semantically independent spans and reports geometric-mean speedups of 1.21–1.93× at a length-controlled win-rate delta of +2.2 % to −7.1 % — the most honest published speed/quality curve in this family. Plato/ASGD [18] replaces the flat list with a **dependency graph** over sub-problems and reports a 68 % throughput gain with a 90 % quality net-win rate against SoT. Hogwild! Inference [19] takes the opposite tack: concurrent workers sharing a live KV cache, and finds that modern reasoning models do this without fine-tuning.

ParallelBench [20] supplies the theory: the conditional-independence assumption underlying parallel generation "inevitably degrad[es] generation quality when dependencies are strong." Tran and Kiela [21] give the information-theoretic version via the data processing inequality, finding single-agent best or statistically tied at every thinking-token budget above the smallest.

What is missing from this literature is my subject: **nobody has combined prompt-level decomposition with dispatch to untrusted volunteer nodes.** That intersection, and not either half of it, is what this document discloses.

2.4 Genome assembly: what the analogy licenses

Lander–Waterman coverage statistics [26] give, for a genome of length G sampled by N clones of length L with minimum detectable overlap fraction θ:

```
c = L·N / G                                  (coverage redundancy)
P(base uncovered)          = e^(−c)
E[# apparent islands]      = N·e^(−c·θ)
E[# clones per island]     = e^(c·θ)
```

with the familiar $\theta \to 0$ simplification `E[contigs] = N·e^(−c)` from which the "8× coverage" rule follows: `e^(−8) ≈ 0.034 %` of bases uncovered.

This model transfers, but only after one correction that earlier drafts of this work got wrong.

The relevant difference between genome assembly and text assembly is **not** that the target sequence is *known*. In *de novo* assembly no reference exists: the sequence is recovered by alignment, probability and biological plausibility checks on the consensus. An earlier version of this paper said "pre-existing" in a way that implied "known", and that was simply an error.

The real difference is narrower and more useful. In genomics there exists **a single physical molecule of which every read is a sample**. That uniqueness is what guarantees that two true overlaps are reconcilable: both reads came from the same object. In free text generation there is no such guaranteeing object — two nodes writing about the same sub-topic are not sampling anything common; they are independently *creating* content that may or may not agree.

There is, separately, a real shared formal substrate: the shortest-common-superstring problem underlies both genome assembly and text reassembly [27]. What does not exist is a transferred algorithm, and the historical direction of technique runs the other way — distributed and high-performance computing has been applied *to* assembly rather than derived *from* it [28].

But the guaranteeing object can be manufactured. If the plan `D` and the global contract `Γ` are fixed *before* any generation occurs and are treated as the reference — a semantic sequence that every fragment samples — then a common underlying object exists again, and coverage statistics become applicable. Section 5.4.1 develops this, and it converts the analogy from a naming convention into a derivation.

There remains one genuine transfer from the assembly literature, and it is a warning. In de Bruijn assembly a repeat longer than k collapses into a single graph node; the Eulerian path stops being unique, and the number of valid reconstructions grows combinatorially with repeat count [29]. Twenty years of practice established the consequence: **repeats, not coverage, are the binding constraint** [30, 31]. Adding depth does not resolve a repeat. Translated: increasing redundancy does not fix an assembly whose fragments are semantically ambiguous with respect to one another, and the failure modes that matter — chimeras, collapses, misassemblies [32] — are structural rather than statistical.

---

3. Design principles

P1 — Cross the network once per unit of work. The only defensible performance argument available to a volunteer network.

P2 — The orchestrator may refuse. A system that fragments every request is strictly worse than SoT was in 2023, because SoT shipped a router. Fragmentation is a decision with an asymmetric cost function, not a default.

P3 — Model dependencies explicitly. Plans are directed acyclic graphs. Parallelism is the width of a level, not the size of the task set.

P4 — Select before you synthesize. Where several candidate fragments exist, choose one and splice it. Rewrite only where a seam actually fails. Section 5.3 gives the evidence; it is the single most counter-intuitive finding in the literature I surveyed.

P5 — Calibrate every threshold. No fixed cosine cutoffs, no assumed redundancy ratios. Thresholds are derived from labelled data per model and per domain, with asymmetric objectives, and re-derived whenever the embedding model changes.

P6 — Report the tax. Every assembly returns a coherence audit. A protocol that hides its own degradation cannot be evaluated, and will not be trusted.

P7 — Verify cheaply or not at all. Verification that costs a significant fraction of inference destroys the economics. Section 9.3 selects schemes with overheads of order 1 %.

P8 — Route by sensitivity, do not pretend to encrypt. Confidentiality is a routing decision with three lanes, not a property claimed for fragmentation.

---

4. The context budget

This section states the paper's central constraint. It is what the earlier development of this project — and, as far as I can tell, the surrounding literature — leaves implicit.

4.1 Definition

Let a request P be decomposed into micro-tasks $T = \{t_1 \dots t_N\}$ with dependency DAG $D = (V, E)$. Each dispatched packet is

```
K_i = ( Γ , σ(R_j : (t_j → t_i) ∈ E) , t_i )
```

where **Γ** is the *global contract* — objective, audience, register, output format, target length, forbidden vocabulary, session identifier — and σ(·) summarizes the results of t_i's predecessors.

Define the **context budget**

```
S = |Γ| + E[ |σ(·)| ]          (tokens of shared context per packet)
```

and the **contextual redundancy ratio**

```
ρ = ( Σ_i |K_i| ) / |P|  ≈  1 + N·S / |P|
```

ρ is what the operator pays; S is what the operator chooses.

4.2 The four-way tension

Four desiderata are functions of S, and they do not agree:

Desideratum	Behaviour in S	Mechanism
Assembly coherence	**increases**	Workers share the decisions that make fragments compatible. Absent a contract, one worker renders the scene in one register and another in a second, and the client inherits incompatible parts [33]
Fragment verifiability	**increases**	A verifier cannot judge whether a fragment is faithful to a specification it was not given
Privacy by decontextualization	**decreases**	Γ *is* the session's objective, audience and constraints. A node holding Γ holds the shape of the request
Required worker capability	plausibly **decreases**	Context supplied in-prompt substitutes for knowledge held in parameters — stated as a hypothesis in Section 5.4, not as a result

And ρ, hence cost, grows approximately linearly in $N{\cdot}S$.

4.3 The falsifiable core

> **Proposition (Context Budget).** Swarmbly is viable if and only if there exists a context budget S^* that simultaneously satisfies: a coherence tax below the application's tolerance; a leakage bound below the user's tolerance for the applicable sensitivity lane; a verification accuracy above the protocol's security requirement; and a worker-capability requirement met by commodity small models — all at a ρ whose cost remains below the value of the aggregated capacity.

This is the whole project stated as one testable claim, and it is why the reference implementation measures a curve rather than demonstrating a system. Each leg has its own experiment: coherence in V0 (Section 11.2), verification in V3, leakage in a dedicated privacy audit, capability substitution in the H2 protocol of Section 5.4.

It also predicts something useful. Because S is shared across all four, **any improvement that raises coherence per token of context is worth more than an improvement that raises coherence per token of output** — it buys progress on privacy and cost simultaneously. This makes contract compression, and not fragment quality, the highest-leverage research direction in the design. I did not expect that when I began, and it is the kind of prediction that makes the framing worth stating formally.

4.4 Why the earlier formulation was inadequate

An earlier version of this design specified a fixed redundancy target (`C_sem > 1.2`, "20 % intentional redundancy") derived by analogy from Lander–Waterman, and a fixed seam threshold (`τ_sem = 0.85`) on embedding cosine similarity.

Both are withdrawn. The first is withdrawn for the three reasons in Section 2.4 and because the step from a ratio to a redundancy percentage holds only if all excess length is flank, which ceases to be true the moment a global contract is introduced. The second is withdrawn because contextual embedding space is anisotropic — randomly chosen words already exhibit high mean cosine similarity [34] — because cosine similarity in regularized models can be "arbitrary and therefore meaningless," determined by the regularization scheme rather than by semantics [35], because no embedding model dominates across task types [36], and because the reference library for this operation deliberately recommends **no threshold at all** and warns that the similarity is asymmetric [37].

They are replaced by measurement. ρ is swept; τ is calibrated from labelled seam and non-seam pairs under an asymmetric objective (Section 8.5). Where the earlier formulation asserted constants, this one specifies procedures for obtaining them.

---

5. Swarm intelligence with small models

5.1 The thesis

Swarmbly dispenses with the monolithic model entirely. No participant holds a shard of a 70B or 400B network. Instead, worker nodes run *complete, independent* small language models — typically 1–8B parameters, quantized, running on consumer GPUs, unified-memory laptops or multi-core CPUs — and each of them receives a decontextualized, atomic micro-task and answers it in isolation. The client runs a small model too, but its competence is a different one: not world knowledge, but *logic and syntax*, in the sense of understanding the request, planning its decomposition, and suturing the returned fragments into a coherent whole.

The claim is that advanced capability need not reside in a single large network, but can be the arithmetic result of coordinating many small ones — the answer emerging, as a genome does, only at assembly.

This is a strong claim. It is also partly supported, partly unsupported, and partly false as usually stated. This section separates the three.

5.2 Coverage and conversion

I propose decomposing swarm performance into two independent factors.

Coverage C is the probability that *at least one* worker response to a given micro-task is acceptable, and **conversion** V is the probability that the orchestrator, given that acceptable fragments exist, selects and assembles them into an acceptable whole.

For a plan of N micro-tasks with per-task coverage C_i and a global conversion factor V:

$$Q_{system} \approx V \cdot \Pi_i C_i$$

The product over i is the uncomfortable term — it is why N cannot grow freely — but the factor that decides the architecture is V.

Coverage scales with the swarm. Conversion does not.

The evidence for the first half is strong. Repeated sampling raises coverage log-linearly over four orders of magnitude of sample count: on SWE-bench Lite with DeepSeek-Coder-V2-Instruct, 15.9 % at one sample rises to 56 % at 250 samples, beating a 43 % single-sample state of the art [38]. More nodes genuinely means more correct fragments exist somewhere in the swarm.

The evidence for the second half is equally strong and is usually overlooked. The same work states that "majority voting and reward models plateau beyond several hundred samples" — coverage keeps rising and the *ability to cash it in* saturates [38]. Judge-based selection over diverse teams achieves an 81 % win rate against a single-model baseline, while homogeneous teams achieve 51.2 % — chance — and produce 100 % ties across 756 verdicts under decoupled judging [39]. And in the study closest to Swarmbly's own architecture, an 8B multi-agent system ties a 32B single agent with tools on GAIA (23.0 vs 23.0) and beats it on AIME (55.0 vs 45.0), running 4.2× faster — but performance is "primarily driven by orchestrator capacity rather than sub-agent capacity," and scaling the sub-agents yields "inconsistent and inefficient" returns [40].

5.3 Three consequences

Three things follow from that asymmetry, and they all point away from where an engineering effort would instinctively be spent.

(a) The client is the ceiling, not the network. The marginal value of the *(N+1)*-th node is bounded above by the orchestrator's ability to select among what already arrives. An engineering budget that buys nodes before it buys a better client-side selector is spending in the wrong order. This inverts the intuition the swarm framing invites, and it is, in my view, the single most actionable conclusion in this paper.

(b) Select; do not synthesize. Judge-based selection beats synthesis-style aggregation by 63.1 percentage points, and

Mixture-of-Agents-style synthesis loses to the plain single-model baseline in 42 of 42 tasks [39]. Note that this stands in direct tension with MoA's own reported results — 65.1 % on AlpacaEval 2.0 against GPT-4 Omni's 57.5 % using only open models [41] — and I flag the disagreement rather than choosing the convenient side. The design resolves it conservatively: selection is the default path, synthesis the exception invoked only at a failed seam, and the protocol records which path each seam took so the question can be settled with data of my own.

(c) Heterogeneity is an asset, not a defect. An earlier version of this design treated the diversity of volunteer hardware and models as a problem to be homogenized. The evidence points the other way: diverse teams reach 81 % win rates where homogeneous teams reach chance, and homogeneous outputs tie 100 % of the time — a selector given identical candidates has nothing to select [39]. Cross-family model pairs have been reported to eliminate over 30 % of errors [42].

The volunteer network's model zoo is therefore the substrate that makes selection work. The protocol should preserve diversity deliberately: dispatch critical fragments to workers of *different* model families, not merely different machines. This is a genuine reversal of the earlier design, and it costs nothing to adopt.

5.4 Where the thesis is unsupported: the atomicity hypothesis

The strongest form of the claim — that a 3B model answering an atomic sub-task matches a frontier model — is not established, and publishing it unqualified would be the paper's most attackable sentence.

What is supported is narrower. A position paper from NVIDIA argues SLMs are "sufficiently powerful" and "inherently more suitable" for agentic sub-tasks that are narrow and repetitive, proposing heterogeneous systems that invoke a large model only where general conversational ability is required — but it is explicitly a discussion piece, not a benchmark study [43]. The 8B-ties-32B result above [40] is real, and it is a *system-level* result that the same paper attributes to the orchestrator rather than the workers.

What is *not* established is any general equivalence. And there is a specific mechanism by which the claim can fail: **a smaller worker requires more context to do the same job.** Knowledge the model does not hold in parameters must be supplied in the prompt. That is the fourth row of the table in Section 4.2, and it is why worker capability belongs in the context-budget tension rather than in a separate discussion. Shrinking the worker is not free; it is paid for in S, and S is paid for in privacy and in cost.

I therefore state it as a hypothesis with a measurement protocol rather than a claim:

> **H2 (Capability substitution).** For micro-tasks that are atomic, well specified and verifiable, there exists a context budget S at which a 3–8B worker's fragment quality is statistically indistinguishable from a frontier model's on the same micro-task; and the required S decreases as worker capability increases.
>
> *Protocol.* Fix a micro-task set spanning the categories of Section 11.1. For each of {3B, 8B, frontier}, sweep S and score fragments with blind pairwise judging. Report the S at which the confidence interval on the win rate crosses 0.5, per category. Report categories where no such S exists in the swept range — those are the categories Swarmbly must refuse.

The complementary hypothesis governs the client:

> **H3 (Conversion).** A client-side selector over $k > 1$ heterogeneous workers recovers a specified fraction of oracle-selection quality, where the oracle picks the best available fragment.
>
> *Protocol.* With $k \in \{1, 2, 3\}$ and forced model-family diversity, compute realized quality against oracle selection. Report the recovery fraction and its dependence on the orchestrator's own model size. A recovery fraction that does not improve with orchestrator size would falsify the premise of consequence (a).

An 8B orchestrator is not obviously adequate for this role, and the literature is discouraging: on a stateful planning task, Llama-3.1-8B-Instruct scored near 0–2 %, and even frontier models looped in 92–100 % of trials when constrained by an external validator [44]. Broader planning benchmarks report the same direction [45]. Small models make good *routers* — cheap routers cut costs by over 85 % on MT-Bench while retaining 95 % of GPT-4 performance [46] — but routing is classification, and planning is not. Conflating the two is a mistake this design deliberately avoids: the router in Section 8.1 is a classifier, and the planner in Section 8.2 is allowed to be a larger model than the router.

5.4.1 A coverage model for semantic assembly

With the plan as reference sequence, the Lander–Waterman machinery applies — and the source of randomness turns out to be exactly where the model's assumptions are satisfied.

Setup. The plan D defines an ordered set of semantic units $U = \{u_1 \dots u_M\}$, fixed before any generation occurs. Each dispatched packet K_i targets a subset $S_i \subseteq U$ — its assigned unit plus whatever flanking units it carries as context. Nominal coverage is

$$c = \left(\Sigma_i |S_i| \right) / M$$

the average number of packets covering a unit.

Where the randomness lives. This is the substantive difference from genomics, and it is what makes the transfer legitimate rather than decorative:

> In genome sequencing, the stochastic element is **where the reads land**. In Swarmbly, placement is deterministic — the orchestrator chooses it. The stochastic element is **which packets come back**.

Volunteer nodes fail, time out, disconnect and return unusable output, independently and at a rate the network can measure. Let p be that per-packet loss probability. Effective coverage is then

```
c_eff  =  c · (1 − p)
```

and the classical results hold with `c_eff` in place of `c`:

```
P(unit u is uncovered)      =  e^( −c_eff )
E[ uncovered units ]        =  M · e^( −c_eff )
E[ assembly islands ]       =  N_p · e^( −c_eff · θ )
```

where θ is the minimum detectable semantic overlap, expressed as the fraction of a packet's units that must be shared with a neighbour for the assembler to align them — the direct analogue of Lander–Waterman's detectable-overlap parameter.

The design equation. Inverting the first result gives a redundancy requirement derived from a stated tolerance instead of assumed:

```
c  ≥  ln(1/ε) / (1 − p)
```

for a target uncovered-unit fraction ε. This replaces the arbitrary threshold of earlier drafts with a table:

Loss rate *p*	**ε = 5 %**	**ε = 1 %**	**ε = 0.1 %**
0.05	c ≥ 3.2	c ≥ 4.8	c ≥ 7.3
0.10	c ≥ 3.3	c ≥ 5.1	c ≥ 7.7
0.20	c ≥ 3.7	c ≥ 5.8	c ≥ 8.6

Since replication is the dominant contributor to c, the practical operating range is **k = 3–5 replicas per critical unit**, with hedged dispatch (Section 7.6) acting as an adaptive mechanism that raises `c_eff` on demand rather than paying for worst-case c on every request. This unifies two mechanisms that earlier drafts treated as unrelated.

Scope, and a claim. The model bounds *availability*: the probability that a semantic unit goes unanswered. It does not model semantic correctness — a unit can be covered by five replicas that all agree and are all wrong, which is why correctness is handled separately by consensus and confidence scoring (Section 8.4b) and by verification (Section 9.3).

Within that scope, I believe this to be **the first coverage model published for semantic assembly**, and it is the point at which the genomic framing stops being a vocabulary and becomes a derivation. It yields a design equation where earlier work in this area had a guess, and it identifies precisely where the analogy holds: not in the sampling, which Swarmbly controls, but in the loss, which it does not. The parameters θ and the unit granularity require empirical calibration for natural language; Section 11 specifies how.

5.5 The honest statement of the swarm thesis

> The swarm supplies **coverage**; the client supplies **conversion**. Additional nodes raise coverage with logarithmic returns and do nothing for conversion. Heterogeneity among nodes is what makes selection possible, and should be preserved rather than engineered away. The system's quality ceiling is set by the client's selector, and the worker size that suffices is a function of the context budget rather than a constant.

This is weaker than "a 3B model equals GPT-4 on atomic tasks." It is also defensible, actionable, and it tells you where to spend.

---

6. Architecture

6.1 Roles

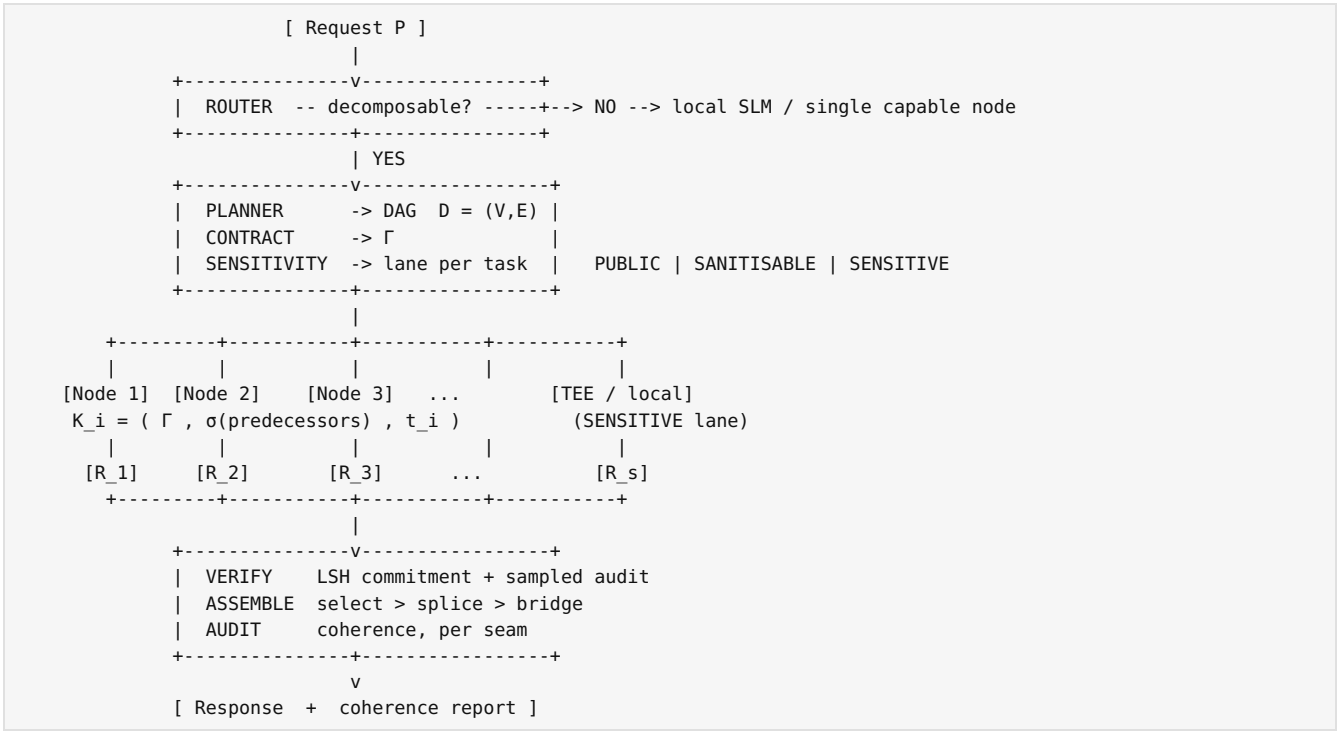
Client / Orchestrator — router, planner, contract generator, sensitivity classifier, packer, speculative dispatcher, verifier, assembler, coherence auditor. Requires an SLM (≥8B recommended; the adequacy of that figure is H3) plus an embedding model.

Worker node — declares a profile, executes micro-tasks, emits verification commitments and telemetry. Runs one complete small model.

Network services — peer discovery (DHT), reputation registry, credit accounting, audit sampler. Deliberately minimal;

no blockchain in v0.2.

6.2 Request lifecycle



Execution proceeds by topological level: all tasks at a level are dispatched concurrently; a level begins when its predecessors have returned and been verified.

6.3 Latency model

The claim `max ≪ Σ` is directionally right and incomplete. The honest model is

```
T_total = T_plan  +  Σ_levels E[ max_{i ∈ level} t_i ]  +  T_verify  +  T_assemble
```

with four terms the naïve version omits.

Two of those terms are local and therefore predictable. Planning scales with `|P|` and runs on the client's small model, and assembly scales with `Σ|Rᵢ|` rather than with the response length, which makes it a cost that grows with the material returned rather than a constant. The other two are set by the swarm, and they are where the model departs from `max ≪ Σ` most sharply.

The first of these is the straggler tail. `E[max]` over W concurrent draws grows with W, and volunteer tails are heavy: the measured effective duty cycle in BOINC is ≈0.61 (0.81 connected × 0.84 active × 0.899 CPU efficiency) and the median host lifetime is 91 days [47, 48]. Retries compound it. With per-node failure probability p, `P(at least one failure) = 1 − (1−p)^W`; at $p = 0.10$ and $W = 20$ that is 88 %, so almost every request retries at least once, and a fixed 10-second timeout would therefore add ≥10 s to the critical path routinely. The protocol mandates **hedged requests** at the p95 of the observed per-class latency distribution instead (Section 7.6).

6.4 Worker profiles and determinism

A worker that runs an undeclared model, or a different quantization, silently changes fragment quality and register. The profile is therefore part of the protocol and is bound to the verification commitment:

```
profile = (model_family, model_version, quantization, prompt_template_id, sampling_params, seed_policy)
```

The orchestrator groups each DAG level into **homogeneous capability classes** to control register mismatch, while deliberately preserving **family diversity across redundant replicas** of the same task (Section 5.3(c)). These two goals are in tension and the protocol makes the trade explicit: homogeneity *within* a fragment's role, diversity *across* candidates for the same fragment.

On worker economics, idle consumer GPUs used for LLM inference have been measured at $0.111–0.149 per million tokens on an RTX 4090, at 62–78 % of H100 throughput for roughly half the cost [49]. This is the figure on which the participation argument rests.

---

7. Protocol specification (v0.2)

This section is written to be implementable. Field names are normative; encodings are given as JSON for clarity and MAY be CBOR on the wire.

7.1 Identifiers

- `session_id` — 128-bit random, generated per request, never reused.
- `task_id` — `BLAKE2b-128(session_id || level_index || task_index)`, truncated to 16 bytes, hex-encoded.
- `attempt_id` — `task_id || ':' || attempt_counter`.

A worker learns `task_id` and `attempt_id`. It MUST NOT learn `session_id`; the derivation is one-way so that two workers cannot determine that they hold fragments of the same session by comparing identifiers. This does not defeat timing correlation (Section 9.2) and is not claimed to.

7.2 Global contract Γ

```
{
  "v": "0.2",
  "objective":  "string  — what the complete response must accomplish",
  "audience":   "string",
  "register":   "formal|neutral|informal|technical",
  "format":     "prose|markdown|json|code",
  "target_len": 0,
  "lexicon":    { "prefer": ["…"], "forbid": ["…"] },
  "entities":   [ { "name": "…", "canonical": "…", "role": "…" } ],
  "style_seed": "string — deterministic style anchor shared by all workers",
  "budget":     { "max_out_tokens": 0 }
}
```

`entities` is the mechanism that prevents inconsistent naming across fragments — the assembly analogue of a shared coordinate system. `style_seed` is a short fixed phrase all workers are instructed to match, which costs a handful of tokens and materially reduces register drift.

`|Γ|` is the dominant term in the context budget S and is therefore the object of the compression research direction identified in Section 4.3.

7.3 Task packet

```
{
  "v": "0.2",
  "attempt_id": "hex",
  "contract": { /* Γ */ },
  "predecessors": [ { "task_id": "hex", "summary": "string", "tokens": 0 } ],
  "task": {
    "instruction": "string",
    "kind": "extract|classify|generate|summarize|transform|judge",
    "expects": { "format": "…", "min_tokens": 0, "max_tokens": 0 }
  },
  "constraints": { "temperature": 0.0, "top_p": 1.0, "stop": ["…"] },
  "commitment_request": { "scheme": "lsh-activation-v1", "params": { "window": 32 } },
  "deadline_ms": 0,
  "lane": "PUBLIC|SANITISABLE",
  "tier": "GLOBAL|TRUSTED",
  "swarm_id": null
}
```

Packets on the `SENSITIVE` lane are never emitted to open nodes; they are executed locally or on an attested TEE endpoint (Section 9.4). The `tier` field is orthogonal to `lane`: it names the *population* a packet may reach rather than the sensitivity of its content, `TRUSTED` requires a non-null `swarm_id`, and packets classified for local-only execution are never serialized as task packets at all (Section 9.5).

7.4 Result

```
{
  "v": "0.2",
  "attempt_id": "hex",
  "text": "string",
  "profile": { "model_family": "…", "model_version": "…", "quantization": "…",
               "prompt_template_id": "…", "sampling_params": { }, "seed": 0 },
  "commitment": { "scheme": "lsh-activation-v1", "digest": "base64", "bytes": 0 },
  "telemetry": { "gen_ms": 0, "queue_ms": 0, "tokens_out": 0, "energy_j": null },
```

```
  "sig": "ed25519 signature over the canonical serialization of all preceding fields"
}
```

`energy_j` is optional and feeds the sustainability accounting of Section 10.3; it is nullable because most consumer hardware cannot report it.

7.5 Node profile advertisement

```
{
  "node_id": "ed25519 public key",
  "models": [ { "family": "…", "version": "…", "quantization": "…",
                "ctx": 0, "tok_per_s_est": 0 } ],
  "capabilities": { "tee": false, "attestation": null },
  "swarm": { "swarm_id": null, "registry": null, "mtls_cert_fingerprint": null },
  "resources": { "vram_mb": 0, "ram_mb": 0 },
  "policy": { "max_tokens_per_task": 0, "kinds": ["extract","generate"] },
  "reputation": { "completed": 0, "audit_pass_rate": 0.0, "since": "ISO-8601" }
}
```

7.6 Dispatch, hedging and retry

1. Filter candidates by tier first — a packet marked `TRUSTED` is offered only to nodes whose public key appears in the whitelist of the named swarm — and then by declared capability, `kind` support, and observed RTT.
2. For a task of criticality `k`, dispatch to `k` nodes selected to **maximize model-family diversity** subject to the capability class.
3. Start a hedge timer at the **p95** of the observed latency distribution *for that task kind and token budget*, not a fixed constant. On expiry, dispatch an additional replica; accept the first result that verifies.
4. Cancel outstanding replicas on acceptance. Record cancellations — a node whose work is habitually cancelled is slow, not dishonest, and reputation must distinguish the two.
5. On verification failure, re-dispatch excluding the failing node and record the event for the audit sampler.

7.7 Parameter table

Parameter	Symbol	Default	Derivation
Context budget	S	swept	Section 4; no default is honest before V0
Redundancy ratio	ρ	measured	Reported, not configured
Seam threshold	τ_sem	**calibrated**	Section 8.5; never a constant
Router threshold	τ_route	calibrated, asymmetric	β<1 in F_β, per SoT-R's Tversky rationale [12]
Criticality replicas	k	1 (3 for critical)	Cost-driven; majority-of-k follows BOINC practice [47]
Hedge trigger	—	p95 per class	Section 6.3
Audit sampling rate	λ	0.01–0.05	Section 9.3; tuned against the penalty schedule [50]
Max plan width	—	8	Straggler tail grows with width (Section 6.3)
Max plan depth	—	4	Beyond this, dependency density argues for not fragmenting

8. Algorithms

8.1 Router

```
function is_decomposable(P) -> (bool, score)
    f ← features(P):
        · task-kind signals (extract / enumerate / summarize vs prove / derive / refactor)
        · length of P
        · density of sequential-dependency markers ("then", "using the result", "step N")
        · presence of shared mutable state (code, ledgers, running totals)
        · request for a single artifact vs a set of items
    score ← classifier(f)                    # 120M-class model is sufficient [12]
    return (score > τ_route, score)
```

τ_route is calibrated with **F_β, β < 1**: a false positive (fragmenting something that should not be) costs more than a false negative. This is the SoT-R lesson, and it is the difference between a system that improves on 2023 and one that regresses from it.

8.2 Planner

```
function plan(P) -> D = (V, E)
```

```
    units ← identify_semantic_units(P)
    for each ordered pair (u, v):
        E ← E ∪ {(u,v)}  if v requires the *result* of u, not merely its statement
    assert acyclic(D)
    if width(D) == 1: return REFUSE       # a chain is not parallelizable; do not pretend
    if depth(D) > max_depth: return REFUSE
    return D
```

The distinction between requiring a predecessor's *result* and requiring its *statement* is the crux. Least-to-most prompting achieves ≥99 % on SCAN against 16 % for chain-of-thought precisely by respecting result-dependencies sequentially [51]; a planner that mistakes a result-dependency for a statement-dependency converts that gain into a loss.

8.3 Packing

```
function build_packet(Γ, t_i, preds, S_target) -> K_i
    base ← Γ                                    # never elided
    budget ← S_target − |Γ|
    order preds by (edge weight, recency)
    for p in preds while budget > 0:
        s ← summarize(p.result, min(budget, cap_per_pred))
        attach(s); budget −= |s|
    return (Γ, attached, t_i)
```

Γ is never trimmed to meet a budget. If `S_target < |Γ|`, the planner reduces N instead: fewer, larger fragments beat many small ones — the LongRAG result, where 4K-token retrieval units and fewer than eight top units matched fully trained state of the art with no training at all [52].

8.4 Assembly

```
function assemble(fragments, D, Γ, τ_sem) -> (text, seam_report)
    ordered ← topological_flatten(D)
    chosen  ← []
    for t in ordered:
        cands ← fragments[t]
        chosen.append( cands[0] if |cands| == 1
                       else judge_select(cands, Γ) )      # select, do not synthesize
    out, seams ← [], []
    for (a, b) in consecutive(chosen):
        sim ← cos( embed(tail_window(a)), embed(head_window(b)) )
        if sim ≥ τ_sem:
            out.append(a); seams.append((a,b,"splice",sim))
        else:
            bridge ← slm.write_transition(tail(a), head(b), Γ)   # exception path
            out.append(a); out.append(bridge)
            seams.append((a,b,"bridge",sim))
    return join(out), seams
```

Every seam is recorded with its similarity and the path taken. The seam report is part of the response, per P6.

8.4b Consensus by multiple alignment of replicas (E16)

Section 8.4 resolves *different* fragments occupying *different* positions. This section resolves *k replicas of the same micro-task*, it is where the genomic analogy pays its most literal dividend, and it produces the one capability in this architecture that a centralized provider cannot replicate at any price.

Two levels, deliberately distinct. Swarmbly assembles at two levels, and conflating them is the error Section 2.4 warns about:

Level	Unit	Mechanism	When
Macro	Different sub-tasks of one large task	Overlap-and-splice with flanking context (Section 8.4)	Long generative work — reports, multi-section documents, corpora
Micro	k complete replicas of the same micro-task	**Multiple alignment and consensus** (this section)	Every micro-task of criticality $k > 1$, including a request that was atomic from the start

An atomic request — one that the router declines to decompose — skips the macro level entirely and goes straight to the micro level with k replicas. **Splitting an atomic question into partial questions is not a supported operation**, because it removes information before sampling rather than sampling redundantly; no amount of coverage recovers it.

Algorithm.

```
function consensus(replicas, Γ, U) -> (text, confidence[])
    # replicas: k complete answers to the SAME micro-task,
    #           from nodes of deliberately different model families (Section 7.6)
```

```
    for r in replicas:
        r.units ← segment_into_semantic_units(r, granularity=U.granularity)

    aligned ← align_multiple(replicas.units)      # progressive alignment on
                                                   # embedding similarity, gaps allowed
    out, conf ← [], []
    for column in aligned:
        agree ← agreement_score(column)            # fraction of replicas whose unit
                                                   # is mutually consistent
        if agree ≥ α_high:
            out.append(majority_unit(column)); conf.append(("HIGH", agree))
        elif agree ≥ α_low:
            out.append(judge_select(column, Γ));   conf.append(("MEDIUM", agree))
        else:
            out.append(judge_select(column, Γ));   conf.append(("LOW", agree))
            flag_low_confidence_region(column)     # surfaced to the user
    return join(out), conf
```

Why this matters beyond assembly. Agreement among *independently sampled replicas from different model families* is a measurable signal about the reliability of a claim. A unit on which five unrelated models converge is not thereby true; a unit on which they diverge is reliably worth flagging. The protocol therefore returns, alongside the answer, a **map of low-confidence regions** — directly analogous to the per-base quality scores that a genome assembler reports instead of emitting a uniformly confident sequence.

No widely deployed commercial system returns this today, and the reason is architectural rather than commercial: **a single model has nothing to align against itself.** Sampling one model repeatedly measures its own variance, not the disagreement between independent estimators. The signal exists here because the network is heterogeneous and distributed, which means the redundancy that decentralization *requires* is the same redundancy that produces the reliability map. A cost of the architecture and a capability of it turn out to be the same mechanism viewed from two directions.

For a user, this is the difference between an answer and an answer that tells you which of its parts to check. For a regulated deployment, it is an audit surface that monolithic inference does not offer.

Three honest caveats.

1. **Agreement is not truth.** Models trained on overlapping corpora share errors. Convergence on a common falsehood is a correlated failure that alignment cannot see. This is precisely why Section 7.6 mandates **cross-family diversity** among replicas: the signal is only as strong as the independence of the samples.
2. **It must be validated, not assumed — and the first attempt to validate it failed.** The correlation between agreement score and factual correctness is an empirical quantity. Measured against a peer-class judge over 597 semantic units, it came back at $r = -0.030$ with flat, non-monotone bins (Section 11.3). That measurement is weak on its own terms — the judge accepted 93.3 % of units, leaving almost no variance for a correlation to appear against — so it leaves the mechanism **unsupported rather than refuted**. Section 11.4 specifies the experiment against ground-truth datasets that would settle it. Until that runs, confidence labels are reported as *agreement*, never as *accuracy*, and the map is not offered as a reliability guarantee.
3. **It costs $k\times$.** Consensus is applied by criticality, not universally.

8.5 Threshold calibration

```
function calibrate_tau(labelled_pairs, embedder, β = 0.5) -> (τ*, curve)
    sims  ← [ cos(embed(a.tail), embed(b.head)) for (a,b,label) in pairs ]
    for τ in quantiles(sims, 200):
        compute precision/recall of "is a broken seam"
        F_β ← (1+β²)·P·R / (β²·P + R)
    return argmax_τ F_β, curve
```

$\beta < 1$ weights precision: declaring a seam broken triggers a rewrite, and unnecessary rewrites are how a system degrades text that was already fine. τ must be re-derived whenever the embedding model changes, for the anisotropy reasons of Section 4.4.

8.6 Coherence audit

Two instruments, reported separately and never merged into a single "quality" score — merging is exactly how the damage hides [12]:

1. **Entity-grid local coherence** [53] — entity mentions and grammatical roles across sentences, scored from transition probabilities. The standard baseline for detecting reordering and insertion damage.
2. **Seam error taxonomy** — the mechanically detectable subset of the error classes identified from 1,193 human annotations over 100 books [54]: entity omission, duplicated content, contradiction, register or tense shift, dangling reference, missing transition, repeated introduction, inconsistent naming. Reported as counts and as the fraction of

sentences free of any detected error.

The headline figure is the **coherence tax**: relative degradation against monolithic generation with the same model.

---

9. Privacy, verification and adversarial nodes

9.1 Fragmentation is not encryption

Earlier development of this concept described decontextualized fragmentation as a form of "quasi-encryption," on the reasoning that a node holding one fragment without global context holds nothing of value. That reasoning does not survive contact with the re-identification literature, and the claim is withdrawn.

Four findings from that literature bear on the reasoning, and they converge. The first is that quasi-identifiers suffice: the combination of ZIP code, date of birth and gender uniquely identifies roughly 87 % of the US population even though none of the three is an identifier on its own [55], and although a later revision puts the figure near 63 %, that is not reassuring. Every syntactic fragmentation or generalization defence proposed in that literature has since been broken by a subsequent attack [56], and sparse high-dimensional data turns out to be inherently re-identifiable from a handful of coarse, noisy attributes [57].

The second is that style is itself an identifier. Authorship attribution operates at internet scale [58], survives shortening and cross-platform domain shift [59], and functions below 280 characters [60]. A fragment "without context" therefore still carries the requester's stylistic signature — and although in Swarmbly's case the fragment is generated by a *worker*, the micro-prompt derived from the user's text is not. The third is that intermediate representations invert: text embeddings reveal almost as much as the text itself [61], and prompts can be recovered from model outputs alone [62].

The fourth is decisive, because it addresses this architecture directly rather than by analogy. In split inference — a client computing part of a network and a server the rest — the ActInv attack achieves precision and recall above 98 % in nearly all evaluated cases, with ROUGE-L consistently above 0.96. Cutting after two client blocks of Qwen3-0.6B yields 99.76 % precision, and even at seven blocks it retains 77.74 %. Defences underperform: at 70 % activation sparsification precision decreases "only modestly," and Gaussian noise at variance 10^{-1} is required before recovery degrades substantially [63].

Swarmbly does not transmit activations, which places it in a better position than split inference. But the direction of the evidence is unambiguous, and there is a further argument internal to this design: **Section 4.2 establishes that coherence requires shipping the global contract Γ to every worker.** A node holding Γ holds the objective, audience, format and constraints of the session. Decontextualization and coherence are antagonists by construction, and no amount of engineering dissolves that.

9.2 What can be claimed

> Swarmbly reduces the exposure surface relative to a centralized provider that reads and retains the complete prompt, and relative to pipeline-parallel schemes in which nodes observe intermediate activations and text under generation. It provides **no cryptographic guarantee of confidentiality**. An adversary controlling a significant fraction of nodes, or correlating by timing and session identifier, can reconstruct a substantial portion of a session.

That is defensible, is still an improvement worth having, and can be said in a user interface without embarrassment.

Two residual channels are worth naming because they are cheap to overlook: **timing correlation** (fragments of one session arrive in a burst; the one-way `task_id` derivation of Section 7.1 does not hide this) and **contract fingerprinting** (a distinctive Γ is itself a session identifier across the nodes that receive it). Mitigations — jittered dispatch, per-node contract paraphrase — cost latency and coherence respectively, which is again Section 4.2.

9.3 Verification

Strong cryptographic confidentiality is unaffordable here, and it is worth stating the numbers so the conclusion is not mistaken for defeatism. General-purpose MPC on a transformer runs at a slowdown of order 10^4–$10^6\times$, with 280.99 GB of communication for a single BERT-Base inference [64, 65]; the best two-party systems report roughly 8 minutes per token for LLaMA-7B [66]. Zero-knowledge proofs of inference need under 15 minutes to prove one forward pass of a 13B model [67]. None of this fits a volunteer economy.

What does fit is a two-layer scheme:

Layer 1 — computational integrity by locality-sensitive commitment. A commitment scheme over activations detects unauthorized model, prompt or precision substitution with 100 % accuracy, zero false positives and zero false negatives in reported testing, at 258 bytes per 32 tokens — roughly 1000× compression against raw embeddings — validating faster than the original inference and remaining robust across GPU types and computation reorderings [68]. This is what makes a node market possible: it costs almost nothing and it closes the obvious fraud, which is a node advertising an 8B model and serving a 1B one.

Layer 2 — sampled public audit. Verification at approximately 1 % of inference cost, secure under a *one-honest-*

verifier assumption rather than an honest majority, with failure probability $P_{fail} \sim \rho^k$ for corruption rate ρ and committee size k. The essential design property: **workers cannot distinguish an audit task from a real one** [69]. The audit rate λ and the penalty schedule are set together; the relationship between sampling rate, penalty size and honest-play equilibrium is formalized in [50].

Layer 3 — selection as a defence. With $k > 1$ replicas, the judge-based selection of Section 8.4 already discards anomalous fragments as a side effect of improving quality [39]. This is the cheapest defence in the system because it is paid for by something else.

What none of these layers do is verify *semantic faithfulness*. Layer 1 proves that a declared model was run on a declared input; it does not prove that the resulting prose is true, or non-malicious. That gap matters because every returned fragment is untrusted input flowing into the client's model — squarely the territory of prompt injection, supply chain, improper output handling and embedding weaknesses in the current application-security guidance [70]. And the client cannot be relied on to notice: off-the-shelf reasoning models attribute failures in agentic systems at under 10 % accuracy [71]. A system that cannot attribute *honest* failures will not detect adversarial ones. The protocol's answer is to constrain the blast radius — fragments are data, never instructions; the assembler runs with output-handling defences; and `kind`-specific output schemas are enforced before a fragment enters the assembly context.

9.4 Sensitivity lanes

Lane	Criterion	Destination	Cost
PUBLIC	No PII, no commercial secret	Open volunteer nodes	None
SANITISABLE	Detectable, pseudonymizable PII	Open nodes; rehydrated locally	Real residual risk (below)
SENSITIVE	Health, legal, financial, identifiable	Local execution, or attested TEE	**<7 % mean overhead** on H100 confidential computing, below 5 % for typical queries and tending to zero as model size grows [72]; independent measurements under Intel TDX report 8.9–21.8 % depending on regime [73]

The TEE lane is what makes the protocol adoptable by an organization, and it is affordable: single-digit percentage overhead is the only confidentiality primitive in this space with that property.

The SANITISABLE lane must be described honestly to users. Against an undefended model on a legal-text corpus, PII extraction reaches ~23 % recall and ~30 % precision, and PII *inference* from 100 candidates reaches 70 %, 50 % and 28 % on three corpora. Differential privacy at $\varepsilon=8$ reduces extraction recall to about 3 % — but not to zero [74], and differentially private generation measurably degrades language quality [75]. Sanitization reduces risk; it does not eliminate it, and the interface should say so rather than bury it.

9.5 Dynamic privacy tiers and trusted swarms (federated topologies)

The lanes of Section 9.4 classify *work*. They say nothing about the *population of machines* that work is permitted to reach, and it is the tacit assumption of a single undifferentiated pool of anonymous volunteers that forces the SENSITIVE lane into the narrow choice between local execution and attested hardware. A second axis is available, and it costs almost nothing to add: the topology itself can be tiered, so that the routing decision is a pair — which lane, and which mesh.

Classification, and where it runs. Every request passes a privacy classifier before planning, and the classifier operates in two modes. The first is a **manual hard flag** — `--privacy=trusted`, `--privacy=local` — which is deterministic, declared by the user, and never overridden by the automatic path; a user who states that a document is confidential is not asking for a second opinion. The second is **automatic triage**: a small local model performs named-entity recognition over the prompt and raises the tier when it detects entities of regulated classes, among them personal identifiers, health and financial data, credentials, and named internal projects.

The essential property is that this classifier runs **entirely on the client**. A privacy classifier that consults the network to decide whether the prompt is private has already disclosed the prompt, and there is no configuration under which that is acceptable; the triage model is therefore part of the client stack, not a service. It is also deliberately recall-oriented — it must over-classify, because the cost of routing a public prompt to a trusted swarm is some throughput, while the cost of the converse is the failure the tier exists to prevent. The honesty of Section 9.4 about sanitization applies here unchanged: automatic triage *raises* a tier, it does not *certify* the absence of sensitive content, and the interface says so.

The three tiers. Tier 1, the global untrusted mesh, is the default and is the network this paper has described up to this point: open volunteer nodes, PUBLIC and SANITISABLE lanes, the full verification stack of Section 9.3, and redundancy at the k derived in Section 5.4.1.

Tier 2, the trusted swarm, is a permissioned sub-mesh. Membership is a cryptographic whitelist of node public keys under an operator-controlled registry; every link carries mutual TLS, so that both endpoints are authenticated rather than only the client; and the typical deployment is a corporate LAN, a campus network or a VPN overlay. The protocol is unchanged — a trusted swarm is the same protocol over a restricted population, not a second protocol — and that

constraint is deliberate. Round-trip times inside such a swarm collapse from tens or hundreds of milliseconds to well under one, which means the bandwidth asymmetry of Section 3, the entire reason for fragmenting the problem rather than the model, is locally suspended. It would be technically possible to run a finer-grained partition inside the firewall. Swarmbly does not, because a deployment that behaves one way inside the perimeter and another outside it is two systems to implement, verify and reason about. What the low latency buys instead is headroom in the context budget S: larger per-fragment context K_i, more generous overlap, a higher redundancy ratio ρ, and therefore better coherence at the same wall-clock — an improvement obtained by spending the same design parameter rather than by introducing a new mechanism.

Tier 3, pure local execution, is selected by `--privacy=local` and means what it says: no packet leaves the machine. Planning, generation and assembly are all performed by the client model, capability is bounded by local hardware, and the degradation relative to Tier 1 is the honest price of the guarantee. This is the only tier in which Swarmbly makes an unconditional confidentiality claim, and it can make it precisely because there is no network to make it about.

Redundancy in a trusted swarm, and what lowering it costs. The parameter k has served two purposes simultaneously throughout this paper, and the trusted-swarm setting is the first context that separates them. **Adversarial redundancy** defends against a node that lies about its work; it is the assumption behind Section 9.3 and behind the Sybil discussion that follows, and it is exactly what a cryptographic whitelist removes, since majority-vote defence against dishonest generation inside a population of authenticated, contractually accountable machines is paying for a threat already eliminated at the identity layer. **Epistemic redundancy** is a different thing: the confidence map of Section 8.4b needs k independently generated replicas from different model families in order to have anything to align, and it is not a defence against anybody. Reducing k to 1 removes the second along with the first.

The protocol therefore permits the reduction but does not permit it silently. A trusted swarm **MAY** set criticality to $k = 1$ for cost or latency, which is a legitimate operating choice; when it does, the response metadata records that no confidence map was produced and the client surfaces that absence explicitly rather than presenting an empty map as agreement. An operator choosing $k = 1$ is choosing throughput over an audit surface, and the specification's role is to make that choice visible, not to make it. Coverage constrains the floor independently: with `c_eff = c(1 − p)` and `c = 1` there is no margin against loss at all, so the design equation of Section 5.4.1 requires $k \geq 2$ whenever the measured intra-swarm loss rate p exceeds the tolerance ε, LAN reliability notwithstanding.

Why the tier matters beyond engineering. Data-protection regimes are written around identifiable, contractually bound processors: the processor relationships of the GDPR, business-associate agreements under HIPAA, and their equivalents elsewhere all presuppose an entity that can be named, audited and held liable. An anonymous volunteer cannot be a processor under any of them. A whitelisted, mutually authenticated node inside an operator's own registry can. Tier 2 is therefore not a performance feature; it is the construction under which this architecture becomes lawful in settings where Tier 1 is not, and it converts "cannot be used with patient data" into "can be used with patient data, on the institution's own machines, under the institution's own registry, with the same client and the same protocol."

And what it does not do. A trusted swarm relocates trust rather than eliminating it. Whoever controls the whitelist controls the swarm, which makes registry governance a security-critical function rather than an administrative one, and a compromised member inside the perimeter is *more* dangerous than an untrusted node outside it — precisely because the redundancy that would have caught it may have been reduced. Mutual TLS authenticates the channel and the identity; it says nothing whatever about whether the model behind that identity is the declared one. For that reason the locality-sensitive commitment of Section 9.3 remains **REQUIRED** inside a trusted swarm even where audit sampling and majority vote are relaxed. A federated topology is a change of threat model, not the absence of one.

9.6 Sybil resistance: a limitation, declared

Without a trusted identity authority, a single adversary can present arbitrarily many distinct identities, defeating **any** scheme based on redundancy or majority vote [76]. Reputation systems do not escape this: the canonical P2P algorithm requires a set of *pre-trusted* peers to be Sybil-resistant, which reintroduces the anchor it was meant to remove [77].

That this is not merely theoretical is visible in the flagship volunteer-computing deployment, where 41.4 % of hosts belonged to single-host users, 44.2 % to users with 2–10 hosts, and the largest single user operated 2,987 hosts [47] — extreme concentration, by a benign participant with no incentive to conceal it.

Swarmbly is therefore not Sybil-resistant in the strong sense, and the protocol says so. It adopts layered trust: accumulated reputation, a cost to enter the registry, sampled audit with economic penalty, and a set of anchor nodes operated by the foundation for cold start. Comparable systems take the same medicine under different names [6].

---

10. Economics, governance and sustainability

10.1 Credits, not tokens

Earlier development proposed a 15 % founder premine and a 0.5 % protocol fee on each micropayment. Both are withdrawn on regulatory and narrative grounds. The Swiss framework classifies tokens as payment, utility or asset, with a two-condition test to escape securities classification [78]; a premined, transferable instrument with an expectation of

appreciation is the archetype that triggers it. European exemptions are narrow — €1,000,000 over twelve months, or 150 persons per member state, with service-provider rules in force since 30 December 2024 [79].

The design that stays outside both regimes is deliberately unexciting: credits that are non-transferable between accounts, earned by processing and spent by requesting; no presale and no premine; immediate utility from day one rather than a promise; expiring balances to discourage hoarding; and fiat conversion in one direction only — enterprises buy capacity through the commercial arm, volunteers do not sell credits. This is less exciting than a token and it is what permits launching without securities counsel in three jurisdictions.

10.2 Licence and governance

The protocol implementation is **AGPL-3.0-or-later**. Its clause 13 closes the network-use gap that GPL leaves open [80]. Three qualifications belong in the record: the obligation attaches to the Program and its modifications rather than to a surrounding proprietary stack; the licence covers software, not the protocol, so a clean-room reimplementation is lawful; and its practical force is deterrence rather than litigation, given that at least one major vendor's published policy bans AGPL code outright.

The strongest empirical guidance available on this choice comes from the recent relicensing wave: four of four projects that hardened their licences produced a successful independent fork, and two of the four subsequently reverted to AGPL [81, 82]. Starting at AGPL and staying there is the position that history supports.

Two structural decisions follow. Dual licensing is rejected — it requires an entity that can sell proprietary exceptions, which is incompatible with a foundation whose mandate is openness; revenue comes from managed service instead. And **trademark, not copyright, is the operative control lever**, following the model of a foundation that governs two word marks and two logos with a documented permitted/approval-required split. Contribution is by DCO sign-off, not CLA, because copyright assignment creates friction precisely with the community this project needs.

On structure: a Swiss *Verein* can be constituted quickly and without minimum capital, which is sufficient to hold rights and receive grants; a *Stiftung* is the right instrument later, when there is a treasury to steward. The claim that a foundation is "unacquirable and unsilenceable" is overstated in any case — foundations are captured through boards, donor dependence and control of repositories and marks. Independence is a practice, not a legal form.

10.3 Sustainability, unclaimed

Data centres consumed 415 TWh in 2024, about 1.5 % of world electricity, with projections to 945 TWh by 2030 [83]. US hyperscale facilities draw from grids measured at 545 gCO_2/kWh against a 370 g national average — 48 % dirtier [84]. Those figures support the *motivation*.

They do not support a claim of net benefit, and I do not make one. Energy per token varies by nearly three orders of magnitude across configurations, and datacenter accelerators achieve the lowest energy per token in the large majority of scenarios; idle draw of 12–90 W is paid in full by a node that is available but unused [85]. Global PUE is 1.54, but hyperscalers operate at 1.09–1.15 against a home's effective ~1.0 — a margin of 9–15 %, not an order of magnitude [86]. Redundancy on top of that is pure overhead, and waste heat is recovered in data centres and essentially never in homes.

The commitment I make instead is procedural: adopt the Software Carbon Intensity standard — `SCI = ((E × I) + M) / R`, ISO/IEC 21031:2024, which explicitly excludes offsets [87] — instrument nodes and client, and **publish the result whatever it shows**. The defensible motivating argument is the embodied-carbon one: extending the service life of hardware that already exists avoids new manufacture, and manufacturing is the majority of a large operator's footprint and growing. That argument deserves measurement, not assertion.

On induced demand. Making a resource cheaper usually increases its total consumption rather than displacing existing use — Jevons' paradox — and a reviewer at a climate fund will raise it. I address it directly rather than hoping it goes unmentioned.

Democratizing inference will very likely induce demand that does not exist today. The claim Swarmbly makes is not that this demand disappears, but that **it is absorbed by hardware that has already been manufactured**. In the centralized model, induced demand is met by building more data centres and buying more accelerators, which is precisely the embodied-carbon term that dominates and is growing. In Swarmbly, induced demand is met by raising the utilization of devices already in the world, so **the marginal embodied carbon of serving it approaches zero**.

Two conditions bound this argument and both are stated rather than assumed. It holds only while **spare capacity exists** — at swarm saturation, growth would again drive new manufacturing. And it holds only for the fraction of traffic served by genuinely pre-existing volunteer hardware, which explicitly **excludes the foundation-operated anchor nodes** of the bootstrap period (Section 10.4). The public dashboard reports that fraction, so the claim can be audited rather than asserted.

10.4 Bootstrap: anchor nodes, declared

A network whose supply and demand must arrive simultaneously does not start on its own. Swarmbly's bootstrap subsidises supply: the foundation operates rented capacity so that service is fast and stable from the first day, and retires it as community supply grows.

This creates an integrity exposure that the protocol handles by disclosure rather than by omission. During that period a

portion of the "volunteer swarm" is rented datacenter hardware, and for that portion **the embodied-carbon argument does not apply and the decentralization claim is only partially true**.

The commitments are therefore explicit:

- Such nodes are named **anchor nodes operated by the Foundation** and are labelled as such in the registry.
- The public dashboard reports, in real time, **the share of traffic served by anchor nodes versus community nodes**.
- The Foundation publishes a target trajectory for that share and reports against it.

The reduction of the anchor share becomes a public, auditable measure of progress toward real decentralization — and a legible metric for a funder. A network that hides its subsidy has a reputational bomb with a timer on it; one that publishes it has an accountability instrument.

---

11. Evaluation

11.1 Task categories

Fit requires five simultaneous attributes: decomposable into genuinely independent sub-tasks; latency-tolerant; token-intensive; weak inter-fragment dependencies; verifiable or non-sensitive content.

Category	Fit	Reasoning
Bulk document processing	**High**	Embarrassingly parallel, no dependencies, latency-tolerant — the profile that made volunteer computing work at all
Synthetic data generation, labelling	**High**	Independent per sample; filter-verifiable; large volume
Code-migration sweeps	**High**	Independent per file; verifiable by compiling and running tests
Evaluation and judging at scale	**High**	Independent; aggregation is a vote, the safest merge available
RAG over large corpora	**Moderate**	Map stage parallel; but fewer, larger units beat many small ones [52]
Long structured reports	**Moderate**	Decomposable by section — and precisely where SoT degrades [12]. Requires a strong Γ and coherence audit
Multi-hop mathematical reasoning	**Poor**	Result-dependencies; sequential decomposition wins here [51]
Code with shared mutable state	**Poor**	The canonical failure through conflicting implicit decisions [33]
Interactive low-latency chat	**Very poor**	Lossless single-node methods already dominate [13, 14, 15]

11.2 V0 — the coherence tax

The reference implementation accompanying this paper implements V0 in full, with no networking. It runs the complete pipeline in one process, sweeping S (reported as ρ) and N, and comparing against two baselines: monolithic generation with the same model, and speculative decoding as the honest single-node comparator.

It reports the entity-grid score, the seam-error taxonomy, an overall judge score kept separate from both, the achieved ρ, per-seam similarity and path, and the resulting coherence tax.

> **Go/no-go.** There must exist a ρ at which coherence degradation is **below 5 % relative to monolithic generation, in at least one task category.** If no such ρ exists, the architecture is not viable for generative assembly, and the project should either stop or restrict itself to workloads with no seam to break — classification, extraction, labelling. The harness prints this verdict on every run.

The intent of stating a stopping rule before collecting data is to make the result informative in both directions.

11.3 First measurements

V0 and the agreement calibration have now been run against real models. What follows is the whole result, including the part that does not support a claim made earlier in this paper.

Setup. Three model families served by a local Ollama — `llama3.2:3b`, `qwen2.5:3b`, `gemma2:2b` — with `nomic-embed-text` for embeddings. Eight prompts across eight categories, one seed, temperature 0. τ_sem was calibrated on **72 labelled pairs** ($F_{0.5}$ = 0.988, precision 1.00, recall 0.944) and came out at **0.51**. The run metadata records that this was not the mock backend and that the embedding route did not degrade; the numbers below are void without both, which is why the harness reports them.

The coherence tax falls monotonically in ρ. BooookScore-like tax against the monolithic baseline, at k = 1, 21 valid cells per ρ:

ρ (target)	ρ (achieved)	Coherence tax	Absolute difference
1.00	1.17	+24.1 %	+0.124
1.25	1.27	+20.4 %	+0.076
1.50	1.53	+16.1 %	+0.068
2.00	2.08	+13.7 %	+0.052

Both the ratio and the denominator-free absolute difference decrease with ρ. This is the behaviour hypothesis H1 predicts and the first evidence that the context budget of Section 4 is the variable the design says it is.

The go/no-go criterion of Section 11.2 is met. Six of 28 category × ρ cells fall below the 5 % threshold fixed before any data existed. `synthetic_data` clears it at every ρ tested (+1.3 %, −5.1 %, −6.2 %, −0.3 %); `creative_writing` clears it at ρ = 2.0 with **−9.0 %**, and `code_shared_state` at ρ = 1.5 with +3.2 %. A negative tax means fragmentation *improved* the answer on that instrument. The criterion was written as "at least one task category" precisely because no one expected all of them to pass, and they do not.

Two measurement failures, reported because they bound what the table above can mean. First, the entity grid is unusable on this corpus: the monolithic baseline for that instrument ranges from 0.000 to 0.114 across all eight prompts, median 0.024. Every one of the 96 cells therefore divides by a near-zero denominator, and all 96 are excluded. The entity-grid coherence tax is **not measured here**, and an early version of this run reported figures as extreme as −180 % before the denominator was checked. Second, one prompt produced a monolithic baseline of a single sentence and six tokens — a generation failure, not a coherence result — and its 12 cells are excluded from the table above.

The confidence map is not supported by the agreement calibration. Sweeping $k \in \{1, 3, 5\}$ at ρ = 1.5 with one replica per family:

k	Coherence tax	Mean agreement	HIGH	LOW
1	+13.2 %	—	—	—
3	+30.8 %	0.577	29.1 %	42.3 %
5	+33.3 %	0.728	58.3 %	28.7 %

Consensus by multiple alignment costs roughly 17 to 20 points of quality relative to $k = 1$, and the per-unit agreement score does not predict judged acceptability: **Pearson r = −0.030 over 597 semantic units.** The agreement bins are flat and non-monotone, and agreement does not order them: the highest-scoring bin is 0.6–0.8, the lowest-agreement bin is second, and the bin where the models agreed most scores below both:

Agreement	Units	Judged acceptable
0.0 – 0.2	40	97.5 %
0.2 – 0.4	91	91.2 %
0.4 – 0.6	80	91.3 %
0.6 – 0.8	122	99.2 %
0.8 – 1.0	264	91.3 %

Section 11.4 states that a flat or negative correlation would invalidate the confidence map as a reliability signal and that such an outcome must be publishable. It is published here.

But this run is not the experiment Section 11.4 specifies, and the difference matters. The judge accepted **93.3 %** of units. With that little variance in the dependent variable a correlation cannot appear even if the underlying signal is real, so this measurement cannot distinguish two very different conclusions: that agreement between independent model families does not predict correctness, or that a peer-class judge does not discriminate quality finely enough to detect it. V3c as specified calls for ground-truth datasets; this run used the judge. **The honest statement is that the confidence map is unsupported, not refuted.**

That distinction does not rescue the claim. An unsupported property cannot be advertised as the architecture's most valuable one, and Section 1.3 has been rewritten accordingly. It does mean the mechanism is not yet dead, and that the experiment which would settle it is now the highest-priority item in Section 11.4.

Scope. Eight prompts, one seed, 2–3B models. This is a smoke-test corpus, not a benchmark, and 2–3B is the low end of the capability range the protocol targets: a result here bounds the architecture at that scale and does not settle it at 8B. These numbers are a signal to act on, not a headline to quote.

11.4 Subsequent phases

V1 — router. Train the decomposability classifier with asymmetric loss; require recovery of ≥80 % of the available gain at under 5 % false-positive rate.

V2 — simulated swarm. Inject churn with measured volunteer parameters: duty cycle 0.61, median host lifetime 91 days [47, 48], wide-area latency distributions [24], failure rates of 5/10/20 %. Require p95 latency within 2× of the failure-free case at $p = 0.10$, $N = 8$, using hedged requests.

V3 — real network, 20–50 nodes. Integrate the commitment scheme [68] and sampled audit [69]. Deliberately inject dishonest nodes: undersized models, plausible fabrications, prompt injection. Require >95 % detection at under 5 % verification overhead.

V3c — agreement calibration. Measure the correlation between the per-unit agreement score of Section 8.4b and factual correctness, against ground-truth datasets, with replicas drawn from deliberately different model families. Report calibration curves per task category. Until this experiment runs, confidence labels are reported as *agreement* and never as *accuracy*; a flat or negative correlation would invalidate the confidence map as a reliability signal, and that outcome must be publishable.

V4 — environmental measurement. Instrument and publish SCI against a centralized baseline.

11.5 Metrics

Metric	Definition	v1.0 target	Measured (Section 11.3)
Coherence tax	Δ seam-free sentence fraction vs monolithic	<5 %	**met in 3 of the 7 categories that produced a measurement**; 13.7 % overall at ρ = 2.0
Operating ρ	Input tokens per prompt token	<2.0	not measured yet
Effective speedup	vs monolithic, same model	>1.5×	not measured yet
Speedup vs honest baseline	vs speculative decoding	Reported even when <1	not measured yet
p95 latency under churn	p=0.10, N=8	<2× failure-free	not measured yet
Dishonest-node detection	Injected adversaries caught	>95 %	not measured yet
Verification overhead	Extra cost per fragment	<5 %	not measured yet
SCI	gCO_2e per functional unit	Published and compared	not measured yet

12. Limitations and negative results

Stated plainly and at length. A specification whose failure modes are documented can be improved by people who did not write it; one that hides them can only be discovered to be wrong. None of what follows retracts Section 1.3 — these are the conditions under which those possibilities do *not* materialise, and they are the agenda for the work that follows publication.

L1 — Quality loss from independent generation is theoretical, not incidental. Parallel generation assumes conditional independence, and quality degrades in proportion to the strength of the real dependencies [20]. At equal compute budget, decomposition is a lossy channel [21]. This cannot be prompt-engineered away; it can only be routed around, which is why Section 8.1 exists.

L2 — Coherence is the axis that breaks, and aggregate scores hide it. SoT improves relevance and diversity while degrading coherence and immersion [12]. Hierarchically merged text exhibits eight recurring, taxonomizable coherence error classes [54]. Any evaluation reporting a single "which is better" judgement will miss the damage.

L3 — The assembler operates in a regime known to be unreliable. Model reliability degrades with input length across all models tested; one distractor hurts and four compound; and models perform *better* on shuffled contexts than on logically coherent ones — meaning an assembler reading semi-related fragments is measurably impaired [88]. Positional bias adds a U-shaped curve in which middle fragments are systematically underweighted [89].

L4 — The context limit is relocated, not eliminated. It moves to the client, which is the weakest node in the system. Hierarchical assembly makes the working-memory requirement grow logarithmically rather than linearly with volume, which raises the practical ceiling substantially — but the ceiling exists, and it is bounded by assembly time as much as by memory.

L5 — An 8B orchestrator may be inadequate. See Section 5.4. This is H3, and a negative result would require a larger client-side requirement, which narrows the addressable user base.

L6 — No strong Sybil resistance. See Section 9.6.

L7 — Fragmentation is not encryption. See Section 9.1.

L8 — Environmental benefit is unproven. See Section 10.3.

L9 — The genomic analogy is a design vocabulary, not an inheritance. See Section 2.4. No genome-assembly algorithm is transferred, and reviewers from bioinformatics should read the analogy as a naming convention plus one transferable warning about repeats.

L10 — The dominant risk is not technical. Volunteer computing has been in decline for two decades: early projects

attracted on the order of a million volunteers, and the user base has since shrunk to roughly two hundred thousand [47]. Swarmbly must explain what makes its incentive loop different, and "network credits" is not by itself an answer. This is, in my assessment, more likely to end the project than any algorithmic limitation.

L11 — Multi-agent systems fail in characterized ways. A taxonomy built from 150 expert-annotated traces (κ = 0.88) and over 1,600 traces across seven frameworks attributes 47.9 % of failures to system design, 32.2 % to inter-agent misalignment and 20.0 % to task verification, with step repetition (15.7 %) and specification disobedience (11.8 %) the most common individual modes [90]. Swarmbly is a multi-agent system and should expect this distribution.

L12 — A trusted swarm relocates trust; it does not remove it. See Section 9.5. Whoever controls the membership whitelist controls the swarm, so registry governance becomes a security-critical function; mutual TLS authenticates an identity, not the model behind it; and a swarm that lowers k to 1 gives up the confidence map of Section 8.4b along with the adversarial redundancy it no longer needs. The tier is a change of threat model, and the specification requires it to be declared rather than assumed.

L13 — The confidence map has no demonstrated value. See Section 11.3. Measured against a peer-class judge, per-unit agreement did not predict judged acceptability ($r = -0.030$ over 597 units), and $k > 1$ cost 17 to 20 points of coherence on the same run. The mechanism is disclosed and specified; its benefit is not established, and the measurement that would establish it has not yet been run. Anyone building on E16 should treat it as an unvalidated hypothesis.

L14 — The second coherence instrument does not function on short answers. See Section 11.3. The entity grid returned monolithic baselines between 0.000 and 0.114 across the whole corpus, which makes every relative comparison built on it a ratio over a near-zero denominator. Either the evaluation corpus moves to longer outputs or the instrument is replaced; until then this paper has one working coherence instrument, not two, and a single mechanical proxy is thinner evidence than the design deserves.

---

13. Prior-art declaration

Published defensively to establish prior art. The elements below are disclosed with the intent that they enter the public domain for patenting purposes; the author reserves copyright in the text under CC BY 4.0 and licenses the implementation under AGPL-3.0-or-later. Each element is disclosed in enabling detail in the section cited.

E1. A method for distributed language-model inference in which the unit of distribution is a **semantic sub-task derived from the request**, dispatched once per fragment per session to nodes each executing a complete independent model, rather than a partition of model parameters requiring per-token network traversal. (Sections 1.2 and 6.2)

E2. A **context budget** S as an explicit protocol parameter jointly governing assembly coherence, fragment verifiability, privacy leakage and required worker capability, with the accompanying redundancy ratio $\rho = \Sigma|K_i|/|P|$ as the reported cost measure. (Section 4)

E3. A **global contract** Γ — objective, audience, register, format, target length, canonical entity table, style seed — transmitted with every fragment as the mechanism for cross-fragment consistency, with the entity table serving as a shared naming coordinate system. (Section 7.2)

E4. A **router with asymmetric decision cost** that may decline to fragment, calibrated with F_β where $\beta<1$ so that erroneous fragmentation is penalized above erroneous refusal. (Section 8.1)

E5. A **dependency-DAG planner** distinguishing tasks requiring a predecessor's *result* from tasks requiring only its *statement*, with parallelism bounded by level width and refusal on degenerate plans. (Section 8.2)

E6. A **packing procedure** in which the global contract is never elided to meet a context budget, and budget shortfall is resolved by reducing fragment count rather than shared context. (Section 8.3)

E7. A **select-then-splice assembler** in which multiple candidate fragments are resolved by selection rather than synthesis, with generative bridging invoked only at a seam whose boundary similarity falls below a calibrated threshold, and with every seam and its path recorded. (Section 8.4)

E8. Empirical calibration of the seam threshold from labelled seam/non-seam pairs under an asymmetric objective, re-derived per embedding model, in place of a fixed cosine constant. (Section 8.5)

E9. A **coherence audit** returned as part of the protocol response, combining entity-grid local coherence with a mechanically detectable seam-error taxonomy, reported separately from any aggregate quality score. (Section 8.6)

E10. Sensitivity-lane routing — PUBLIC / SANITISABLE / SENSITIVE — as the confidentiality mechanism, with the sensitive lane bound to local execution or attested trusted execution, in place of any claim that fragmentation provides confidentiality. (Section 9.4)

E11. A **two-layer verification scheme** for untrusted workers combining a locality-sensitive commitment over activations bound to a declared node profile, with sampled public audit indistinguishable from real work, and redundancy applied by fragment criticality rather than uniformly. (Sections 9.3, 7.4 and 7.5)

E12. Diversity-preserving redundant dispatch: replicas of a critical fragment are assigned to nodes of deliberately *different* model families, on the basis that selection quality depends on candidate diversity, while capability classes are held homogeneous within a fragment's role. (Sections 5.3(c) and 7.6)

E13. Hedged dispatch at a per-class p95 latency trigger with cancellation accounting that distinguishes slow nodes from dishonest ones in reputation. (Sections 7.6 and 6.3)

E14. A **non-transferable, non-premined, expiring credit** accounting unit earned by verified fragment processing and spent on requests, with unidirectional fiat conversion through a commercial service layer. (Section 10.1)

E15. The **coverage/conversion decomposition** $Q \approx V \cdot \Pi_i C_i$ as a design instrument for swarm inference, with the corollary that additional nodes raise coverage with diminishing returns while conversion is bounded by the client-side selector. (Sections 5.2 and 5.3)

E16. Consensus by multiple alignment of independently generated replicas, in which k complete responses to the same micro-task, produced by nodes of deliberately different model families, are aligned at semantic-unit granularity to yield a consensus output together with a **per-unit agreement score**, and in which units below an agreement threshold are surfaced to the user as low-confidence regions — the reliability map being a direct product of the redundancy that decentralization requires, and analogous to per-base quality in a genome assembly. Disclosed as a **mechanism**: its correlation with correctness was measured and came back flat (Section 11.3), so no reliability benefit is claimed for it here. (Sections 8.4b and 11.3)

E17. A **coverage model for semantic assembly** in which the pre-generation plan and global contract serve as the reference sequence, packet loss rather than sample placement is the stochastic element, and the redundancy requirement is derived as $c \geq \ln(1/\varepsilon)/(1-p)$ from a stated tolerance ε for uncovered semantic units at measured node-loss rate p. (Section 5.4.1)

E18. Dynamic privacy tiering with trusted swarms: a client-side privacy classifier — manual hard flag with precedence, plus recall-oriented local named-entity triage that never leaves the machine — driving a routing decision on an axis orthogonal to content sensitivity, into a global untrusted mesh, a permissioned sub-mesh whose membership is a cryptographic public-key whitelist under mutual TLS and an operator-controlled registry, or purely local execution; with the same protocol and the same client across all three, with the latency headroom of the permissioned sub-mesh spent on the context budget rather than on finer-grained partitioning, and with the reduction of replica count inside a trusted swarm permitted only under an explicit declaration that no confidence map was produced. (Section 9.5)

---

14. Conclusion

The knowledge required to build language models is public. The capital required to operate them is not, and that asymmetry — not any secret — is what concentrates control over a general-purpose technology. Meanwhile the hardware capable of serving inference sits idle in hundreds of millions of homes and offices, already manufactured, already drawing power.

The obstacle between those two facts is physical and specific: four to five orders of magnitude between datacenter interconnect and consumer links, which every existing peer-to-peer inference design crosses on every generated token. This paper's structural claim is that crossing it **once per unit of work instead of once per token** is a different regime rather than an optimization of the same one, and that this is what makes volunteer participation possible at all.

Three consequences follow that I did not anticipate when this work began, and that I regard as the paper's substantive contributions — with the third now carrying a measured caveat. The **context budget** unifies four design properties — coherence, verifiability, privacy and required worker capability — as functions of a single scalar, making the protocol's viability one falsifiable proposition rather than four arguments. The **coverage model** turns the genomic framing into a derivation by locating the randomness where the classical assumptions actually hold: in which packets return, not in where samples land. And **consensus by multiple alignment** produces a reliability map that centralized inference cannot generate, because the redundancy decentralization demands is precisely what makes independent estimators available to compare. That third one, **consensus by multiple alignment**, has since been measured, and the first measurement did not support it: agreement between families showed no relationship to judged quality, and $k > 1$ cost quality outright (Section 11.3). The mechanism stands as disclosed; the benefit does not, and the paper now says so wherever it previously claimed otherwise. A fourth contribution arrived later and is smaller in theory but larger in consequence: **privacy tiering**, under which the same protocol runs over an open mesh, a permissioned whitelist under mutual TLS, or nothing but the local machine — because an architecture that cannot be operated lawfully on regulated data is not an alternative to anything, however good its numbers.

The case against is equally specific and is stated at length in Section 12. Independent generation loses quality for reasons that are theoretical rather than incidental. The client-side model on which the design depends sits at the weak end of measured planning ability. Volunteer computing has been contracting for twenty years, and no incentive design in this paper is yet a demonstrated answer to why that reverses.

Both cases are real, and neither is settled by argument. What settles it is a measurement. The first one is now in: against three local model families the coherence tax falls monotonically as the context budget rises, and the abandonment

threshold fixed in advance is cleared in three task categories — while the confidence map, the property this paper was proudest of, came back with no measurable relationship to quality. What remains to settle is whether a context budget exists that satisfies coherence, privacy, verifiability and worker capability simultaneously, at a cost below the value of the aggregated capacity. I have specified the protocol in enough detail to implement, stated the hypotheses that would falsify it, published the harness that measures the first of them, and committed in advance to the threshold at which I would conclude the design does not work.

If it does work, the result is not a cheaper way to buy what is already sold. It is inference capacity that grows with the number of people who participate rather than with the amount of capital available to build, held under a licence and a governance structure designed so that no single party can enclose it. That is worth attempting even at a substantial probability of failure, and it is worth attempting in the open, where it can be checked.

References

> **Verification status and citation style.** In-text citation uses the numeric markers `[1]`...`[90]`; the numbering is unchanged from earlier versions of this document. The reference list below is formatted in **APA 7th edition**. Identifiers marked ⚠ could not be confirmed against a primary source during preparation and **must be completed or verified before submission**; those entries are deliberately left visibly incomplete rather than filled in from memory. Entries whose author list is abbreviated as "et al." carry forward the attribution recorded in the earlier draft and must be expanded to the full APA author list before submission. Everything unmarked was checked against the publisher or arXiv abstract page. Full annotations, including quantitative findings, are in the project's bibliography (`docs/REFERENCES.md`).

Decentralized inference and training

[1] Borzunov, A., et al. (2023). Petals: Collaborative inference and fine-tuning of large models. *Association for Computational Linguistics (ACL) 2023: System Demonstrations*. https://arxiv.org/abs/2209.01188

[2] Borzunov, A., et al. (2023). Distributed inference and fine-tuning of large language models over the internet. *arXiv*. https://arxiv.org/abs/2312.08361

[3] Petals project. (2023). *Petals project repository, release v2.2.0* [Computer software].

[4] Ryabinin, M., & Gusev, A. (2020). Towards crowdsourced training of large neural networks using decentralized mixture-of-experts. *Advances in Neural Information Processing Systems*, *33*, 3659–3672.

[5] Ryabinin, M., Dettmers, T., Diskin, M., & Borzunov, A. (2023). SWARM parallelism: Training large models can be surprisingly communication-efficient. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 29633–29654.

[6] Bittensor. (n.d.). *Incentivizing intelligence*. Bittensor. https://bittensor.com/academia

[7] *Stake-concentration analysis of Bittensor subnets*. (n.d.). ⚠ Author list, year and venue unverified — complete before submission.

[8] Douillard, A., et al. (2023). DiLoCo: Distributed low-communication training of language models. *arXiv*. https://arxiv.org/abs/2311.08105

[9] Jaghouar, S., et al. (2024). OpenDiLoCo. *arXiv*. https://arxiv.org/abs/2407.07852

[10] Jaghouar, S., et al. (2024). *INTELLECT-1 technical report*. arXiv. https://arxiv.org/abs/2412.01152

[11] *Protocol/Subspace Networks*. (n.d.). ⚠ Author list, year, current title and identifier unverified — complete before submission.

Parallel decoding and decomposition

[12] Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., & Wang, Y. (2024). Skeleton-of-thought: Prompting LLMs for efficient parallel generation. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.15337 *(Includes SoT-R.)*

[13] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2211.17192

[14] Cai, T., et al. (2024). Medusa. *arXiv*. https://arxiv.org/abs/2401.10774

[15] Fu, Y., et al. (2024). Break the sequential dependency of LLM inference using lookahead decoding. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2402.02057

[16] Liu, M., et al. (2024). APAR: LLMs can do auto-parallel auto-regressive decoding. *arXiv*. https://arxiv.org/abs/2401.06761

[17] Jin, T., et al. (2025). Learning to keep a promise (PASTA). *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2502.11517

[18] Jin, S., Wu, Y., Zheng, H., Zhang, Q., & Lentz, M. (2024). Adaptive skeleton graph decoding. *arXiv*. https://arxiv.org/abs/2402.12280

[19] Rodionov, G., et al. (2025). Hogwild! Inference. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2504.06261

[20] Kang, W., Galim, K., Oh, S., et al. (2026). ParallelBench: Understanding the trade-offs of parallel decoding in diffusion LLMs. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2510.04767

[21] Tran, H., & Kiela, D. (2026). Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv*. https://arxiv.org/abs/2604.02460

[51] Zhou, D., et al. (2023). Least-to-most prompting enables complex reasoning in large language models. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2205.10625

[52] Jiang, Z., et al. (2024). LongRAG. *arXiv*. https://arxiv.org/abs/2406.15319

Network and hardware

[22] NVIDIA. (n.d.). *NVIDIA H100 product documentation* (NVLink 900 GB/s SXM; PCIe Gen5 128 GB/s). NVIDIA Corporation.

[23] NVIDIA. (n.d.). *NVIDIA Quantum-2 InfiniBand documentation* (400 Gb/s per port; 51.2 Tb/s aggregate). NVIDIA Corporation.

[24] Sevilla, J. (2025). *How far can decentralized training over the internet scale?* Epoch AI. [Read together with Microsoft Azure inter-region latency statistics.]

[25] *Analysis of model-parallel schemes at public-internet latency.* (n.d.). ⚠ Author list, year, title and venue unverified — complete before submission.

Genome assembly

[26] Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics, 2*(2), 231–239. https://doi.org/10.1016/0888-7543(88)90007-9

[27] Khadiev, K., & Safina, L. (2024). Quantum algorithms for the shortest common superstring and text assembling problems. *Quantum Information and Computation*, *24*(3–4), 267–294. https://doi.org/10.26421/QIC24.3-4-2

[28] *Survey of distributed and HPC genome assembly.* (n.d.). ⚠ Author list, year, title and venue unverified — complete before submission.

[29] Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences, 98*(17), 9748–9753.

[30] Nagarajan, N., & Pop, M. (2013). Sequence assembly demystified. *Nature Reviews Genetics, 14*, 157–167. https://doi.org/10.1038/nrg3367

[31] Kingsford, C., Schatz, M. C., & Pop, M. (2010). Assembly complexity of prokaryotic genomes using short reads. *BMC Bioinformatics*.

[32] Chaisson, M. J. P., Wilson, R. K., & Eichler, E. E. (2015). Genetic variation and the de novo assembly of human genomes. *Nature Reviews Genetics*.

Multi-agent systems, selection and aggregation

[33] Yan, W. (2025). *Don't build multi-agents*. Cognition engineering blog.

[38] Brown, B., et al. (2024). Large language monkeys: Scaling inference compute with repeated sampling. *arXiv*. https://arxiv.org/abs/2407.21787

[39] Maryanskyy, A., Budnikov, D., & Kaliyev, A. T. (2026). When agents disagree: The selection bottleneck in multi-agent LLM pipelines. *arXiv*. https://arxiv.org/abs/2603.20324

[40] Żywot, A., Chen, Y., Yuan, S., Søgaard, A., & de Rijke, M. (2026). Can small agents collaborate to beat a single large language model? *arXiv*. https://arxiv.org/abs/2601.11327

[41] Wang, J., et al. (2025). Mixture-of-agents enhances large language model capabilities. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2406.04692

[42] Chen, Y., Niu, G., Cheng, J., Han, B., & Sugiyama, M. (2025). When and why does multi-agent debate fail, and does it really underperform? *arXiv*. https://arxiv.org/abs/2510.20963

[90] Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? *arXiv*. https://arxiv.org/abs/2503.13657

[71] *AgenTracer*. (2025). arXiv. https://arxiv.org/abs/2509.03312 ⚠ Author list unverified — complete before submission.

Small models, planning and routing

[43] Belcak, P., et al. (2025). Small language models are the future of agentic AI. *arXiv*. https://arxiv.org/abs/2506.02153 *(Position paper.)*

[44] Schepanowski, C., & Ling, C. (2025). On the limits of innate planning in large language models. *arXiv*. https://arxiv.org/abs/2511.21591

[45] Valmeekam, K., et al. (2022). PlanBench. *arXiv*. https://arxiv.org/abs/2206.10498

[46] Ong, I., et al. (2024). RouteLLM. *arXiv*. https://arxiv.org/abs/2406.18665

Embeddings and coherence

[34] Ethayarajh, K. (2019). How contextual are contextualized word representations? *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/1909.00512

[35] Steck, H., et al. (2024). Is cosine-similarity of embeddings really about similarity? *Companion Proceedings of the ACM Web Conference (WWW '24)*. https://arxiv.org/abs/2403.05440

[36] Muennighoff, N., et al. (2023). MTEB: Massive text embedding benchmark. *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. https://arxiv.org/abs/2210.07316

[37] Sentence-Transformers. (n.d.). *Semantic similarity and paraphrase mining* [Documentation]. Sentence-Transformers.

[53] Barzilay, R., & Lapata, M. (2008). Modeling local coherence: An entity-based approach. *Computational Linguistics, 34*(1).

[54] Chang, Y., et al. (2024). BooookScore. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.00785

[88] Chroma. (2025). *Context rot* [Technical report]. Chroma.

[89] Liu, N. F., et al. (2023). Lost in the middle. *Transactions of the Association for Computational Linguistics (TACL)*. https://arxiv.org/abs/2307.03172

Verification, privacy and security

[50] Zhang, Y., Wang, S., Liu, X., Tan, S., Popa, R. A., & Moallemi, C. C. (2024). Proof of sampling: A Nash equilibrium-secured verification protocol for decentralized systems. *arXiv*. https://arxiv.org/abs/2405.00295

[55] Sweeney, L. (2002). k-Anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10*(5), 557–570.

[56] Machanavajjhala, A., et al. (2007). ℓ-Diversity: Privacy beyond k-anonymity. *ACM Transactions on Knowledge Discovery from Data, 1*(1).

[57] Narayanan, A., & Shmatikov, V. (2008). Robust de-anonymization of large sparse datasets. *IEEE Symposium on Security and Privacy (S&P)*.

[58] Narayanan, A., et al. (2012). On the feasibility of internet-scale author identification. *IEEE Symposium on Security and Privacy (S&P)*.

[59] *Cross-domain authorship attribution*. (2016). *Privacy Enhancing Technologies Symposium (PETS)*. ⚠ Author list unverified — complete before submission.

[60] *Forensic authorship analysis of microblogging texts*. (2020). arXiv. https://arxiv.org/abs/2003.11545 ⚠ Author list unverified — complete before submission.

[61] Morris, J. X., et al. (2023). Text embeddings reveal (almost) as much as text. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2310.06816

[62] Zhang, C., et al. (2024). Extracting prompts by inverting LLM outputs. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[63] Fan, M., Liu, Y., Wang, F., & Chen, C. (2026). What does the server see? Understanding privacy leakage from large language models in split inference. *arXiv*. https://arxiv.org/abs/2605.23158

[64] Keller, M. (2020). MP-SPDZ: A versatile framework for multi-party computation. *ACM Conference on Computer and Communications Security (CCS)*.

[65] Hao, M., et al. (2022). Iron: Private inference on transformers. *Advances in Neural Information Processing Systems (NeurIPS)*.

[66] Lu, W., et al. (2025). BumbleBee: Secure two-party inference framework for large transformers. *Network and Distributed System Security Symposium (NDSS)*.

[67] Sun, H., Li, J., & Zhang, H. (2024). zkLLM: Zero knowledge proofs for large language models. *ACM Conference on Computer and Communications Security (CCS)*. https://arxiv.org/abs/2404.16109

[68] Ong, J., et al. (n.d.). *TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference*. ⚠ Year, venue and identifier unverified — complete before submission.

[69] *VeriLLM: A lightweight framework for publicly verifiable decentralized inference*. (2025). arXiv. https://arxiv.org/abs/2509.24257 ⚠ Author list unverified — complete before submission.

[70] OWASP Foundation. (2025). *OWASP top 10 for LLM applications*. OWASP Foundation.

[72] *Confidential computing on NVIDIA Hopper GPUs: A performance benchmark study*. (n.d.). ⚠ Author list, year and identifier unverified — complete before submission.

[73] *Benchmarking confidential GPU inference on NVIDIA H100 under Intel TDX*. (n.d.). ⚠ Authorship, year and identifier unverified — complete before submission.

[74] Lukas, N., et al. (2023). Analyzing leakage of personally identifiable information in language models. *IEEE Symposium on Security and Privacy (S&P)*.

[75] *Differentially-private text generation degrades output language quality*. (2025). arXiv. https://arxiv.org/abs/2509.11176 ⚠ Author list unverified — complete before submission.

[76] Douceur, J. R. (2002). The Sybil attack. *International Workshop on Peer-to-Peer Systems (IPTPS)*. https://doi.org/10.1007/3-540-45748-8_24

[77] Kamvar, S. D., Schlosser, M. T., & Garcia-Molina, H. (2003). The EigenTrust algorithm for reputation management in P2P networks. *International World Wide Web Conference (WWW)*.

Volunteer computing

[47] Anderson, D. P. (2019). BOINC: A platform for volunteer computing. *Journal of Grid Computing*. https://arxiv.org/abs/1903.01699

[48] Anderson, D. P., & Fedak, G. (2006). The computational and storage potential of volunteer computing. *IEEE International Symposium on Cluster Computing and the Grid (CCGrid)*.

[49] *Idle consumer GPUs versus enterprise GPUs for LLM inference*. (2025). *ACM AIBC*. ⚠ Author list and identifier unverified — complete before submission.

Licensing, governance, energy

[78] Swiss Financial Market Supervisory Authority. (n.d.). *Guidelines for enquiries regarding the regulatory framework for initial coin offerings*. FINMA.

[79] European Parliament & Council of the European Union. (2023). *Regulation (EU) 2023/1114 on markets in crypto-assets (MiCA)*.

[80] Free Software Foundation. (2007). *GNU Affero General Public License, version 3* (clause 13). Free Software Foundation.

[81] *Redis relicensing to AGPLv3 (May 2025); Elastic adding AGPLv3 (August 2024)*. (2024–2025). ⚠ Author, publisher and source documents unverified — complete before submission.

[82] *Comparative analysis of the 2021–2025 relicensing wave*. (n.d.). ⚠ Secondary source; author, year and venue unverified — complete before submission.

[83] International Energy Agency. (2025). *Energy and AI*. International Energy Agency.

[84] *Facility-level study of US hyperscale data centre grid carbon intensity*. (2026). ⚠ Author list, venue and identifier unverified — complete before submission.

[85] *Energy-aware LLM inference benchmark*. (2026). ⚠ Author list, venue and identifier unverified — complete before submission.

[86] Uptime Institute. (2025). *Global data center survey 2025*. Uptime Institute.

[87] Green Software Foundation. (2024). *Software carbon intensity (SCI) specification* (ISO/IEC 21031:2024). Green Software Foundation.

Document version 1.4, 14 August 2026. Spanish-language companion: `WHITEPAPER_ES.md`. Protocol specification: `SPEC_EN.md`. Reference implementation and coherence-tax harness: `swarmbly_v0/`. Dated public record: Zenodo `10.5281/zenodo.21956743`, `10.5281/zenodo.21957088`, and this repository at tag `v1`.